

Deep Learning for Image Analysis

Mehyar MLAWEH



Master BDIA - Dauphine Tunis
PhD student - Université PSL



Generative AI Engineer - Wealthy Technology

1. **Computer Vision Foundations**
2. **CNNs**
3. **Transfer Learning**
4. **Vision Transformers**
5. **Object Detection**
6. **Segment Anything Model (SAM)**
7. **Final Project**

Lecture 2 - Part 1:

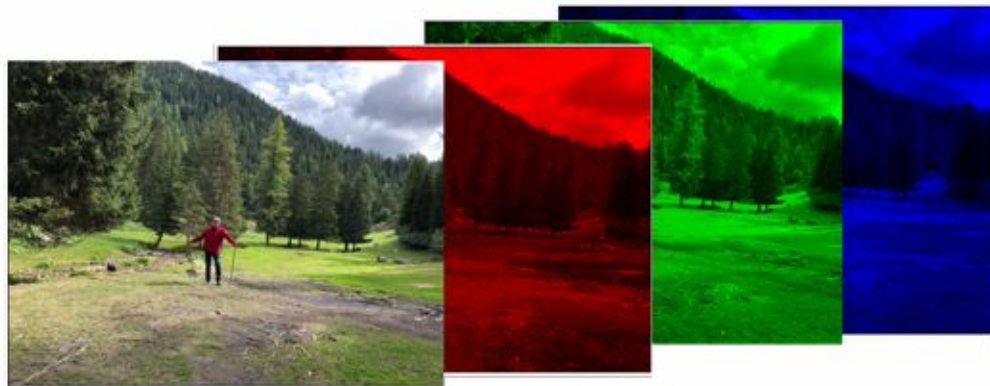
Convolutional neural networks

What is an image?

An image is a matrix of numbers are called pixels Grayscale

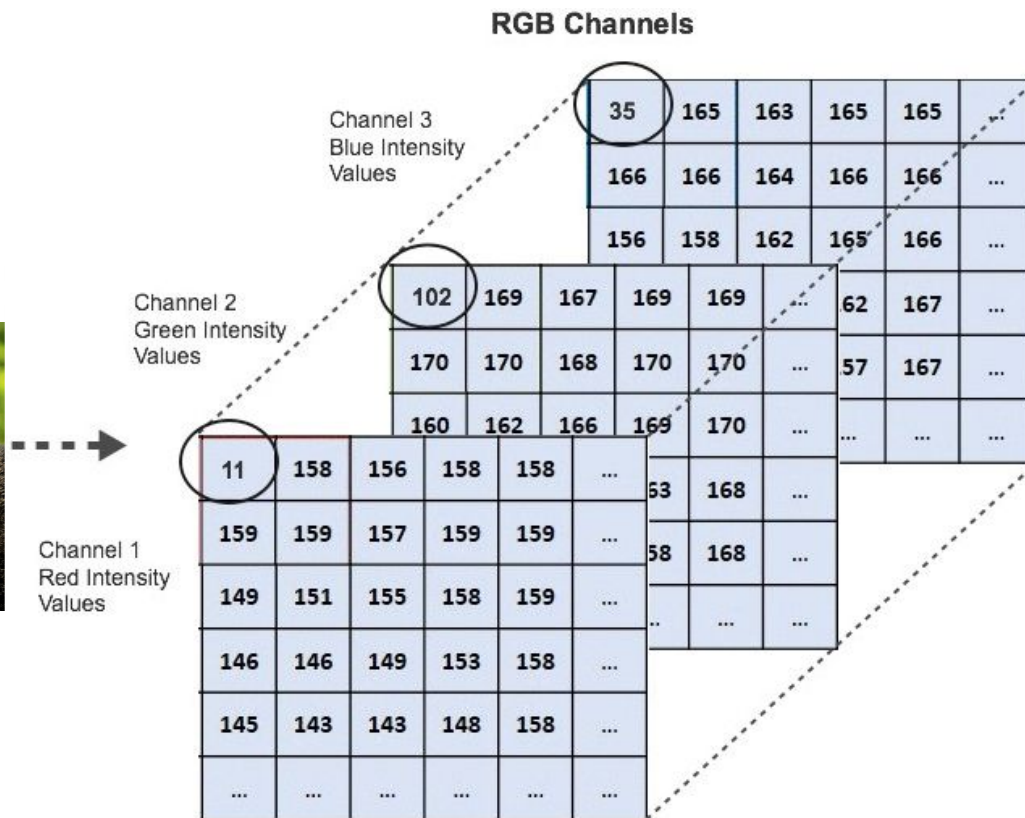
3 channels RGB : Superposition of 3 matrices of numbers between **0** and **255**

- A **larger** value means a **brighter** pixel.
- A **smaller** value means a **darker** pixel.



What is an image?

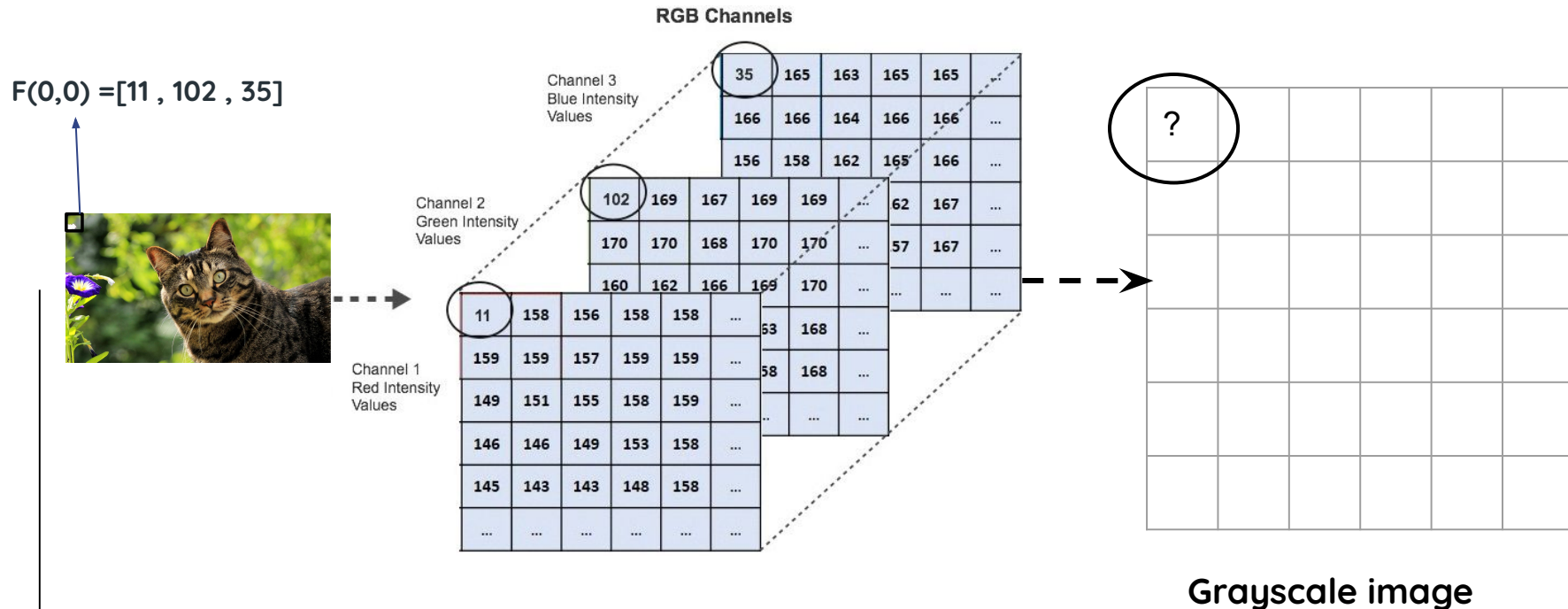
$F(0,0)$



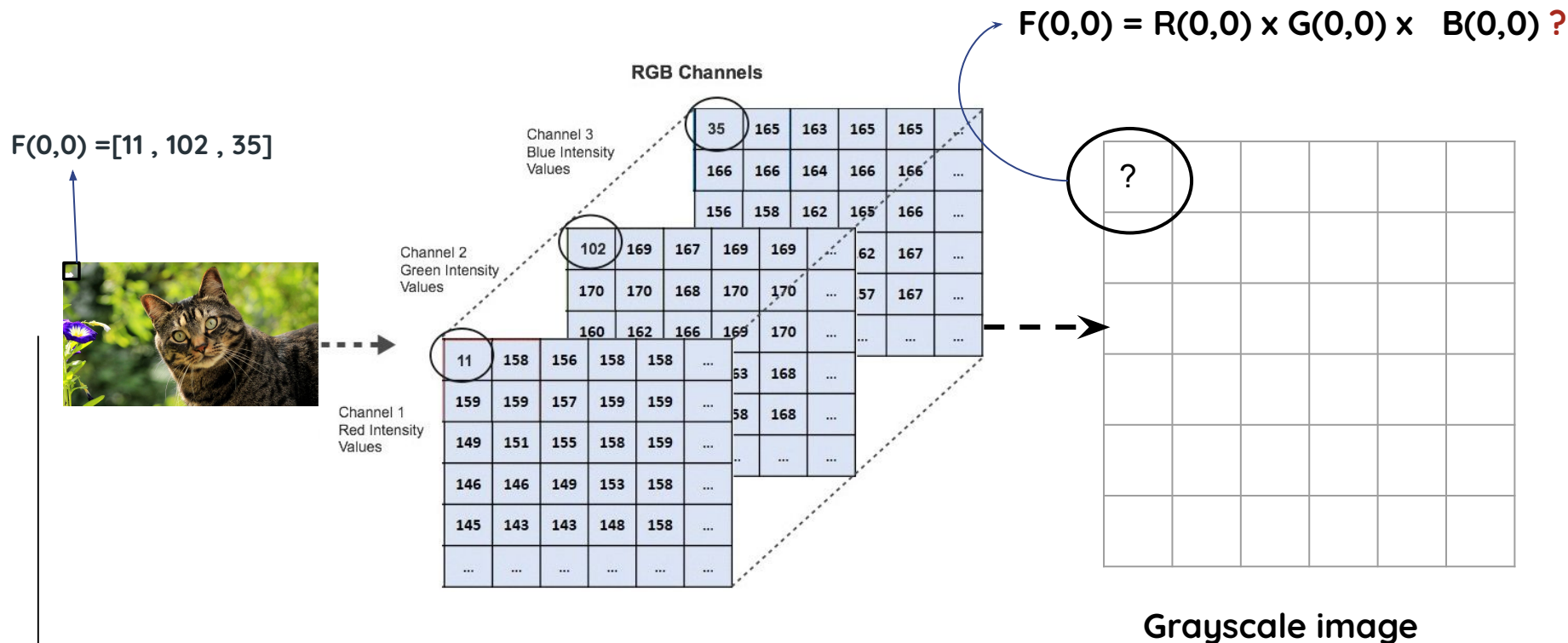
[source: <https://freecontent.manning.com/the-computer-vision-pipeline-part-2-input-images/>]

© Mehya Mlaweh, 2026. All rights reserved.

What is an image?



What is an image?



What is an image?

Grayscale formula

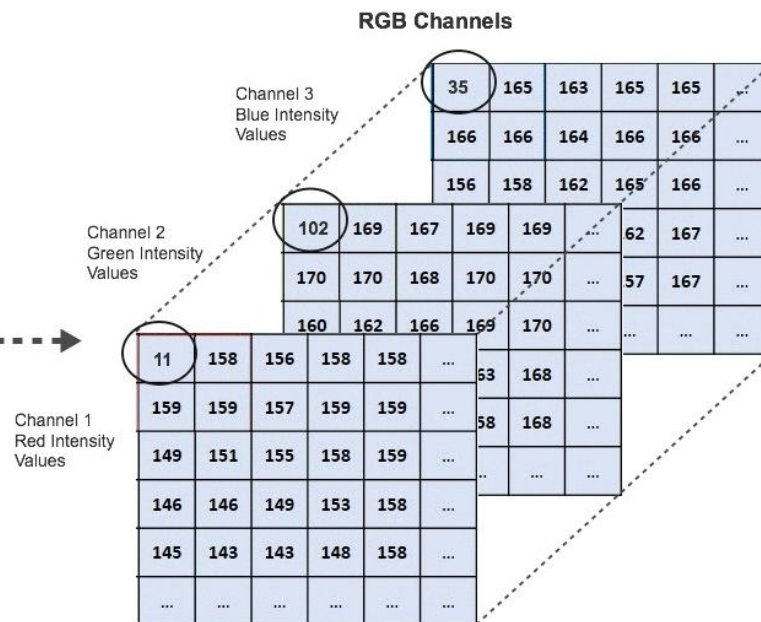
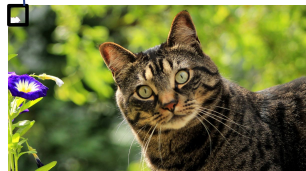
$$\text{Gray} = 0.299.\text{R} + 0.587.\text{G} + 0.114.\text{B}$$

[source: <https://freecontent.manning.com/the-computer-vision-pipeline-part-2-input-images/>]

© Mehya Mlaweh, 2026. All rights reserved.

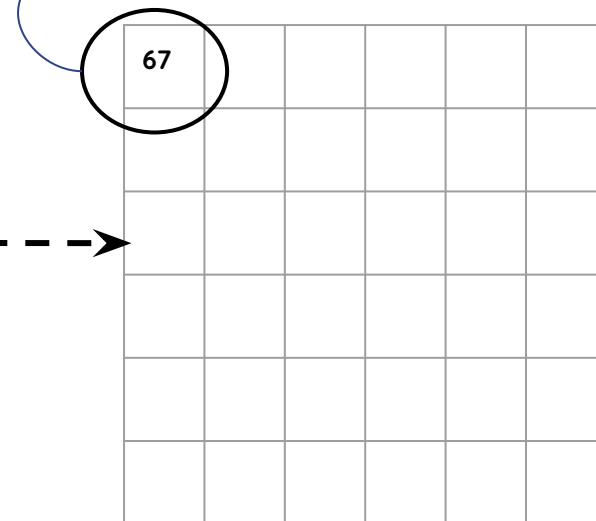
What is an image?

$F(0,0) = [11, 102, 35]$



$$F(0,0) = 0.299 \times 11 + 0.587 \times 102$$

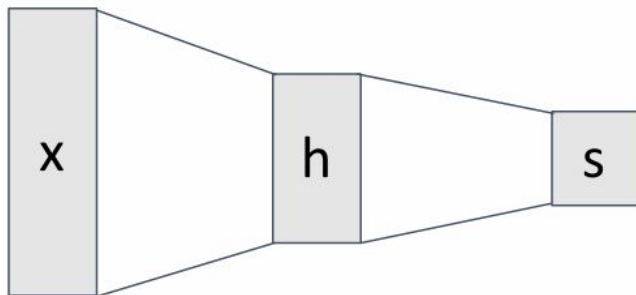
$$+ 0.114 \times 35 = 67.15$$



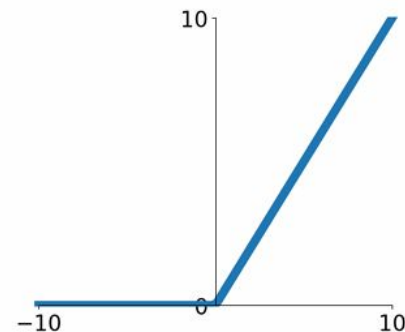
Grayscale image

Fully-Connected Neural Network

Fully-Connected Layers



Activation Function

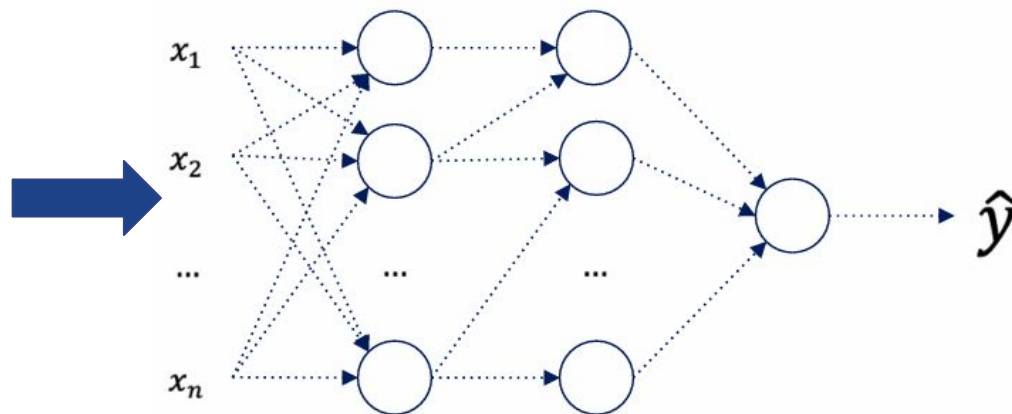




Input image dimension:

2000 x 1000 x 3

(Height x Width x Channels)



Input layer

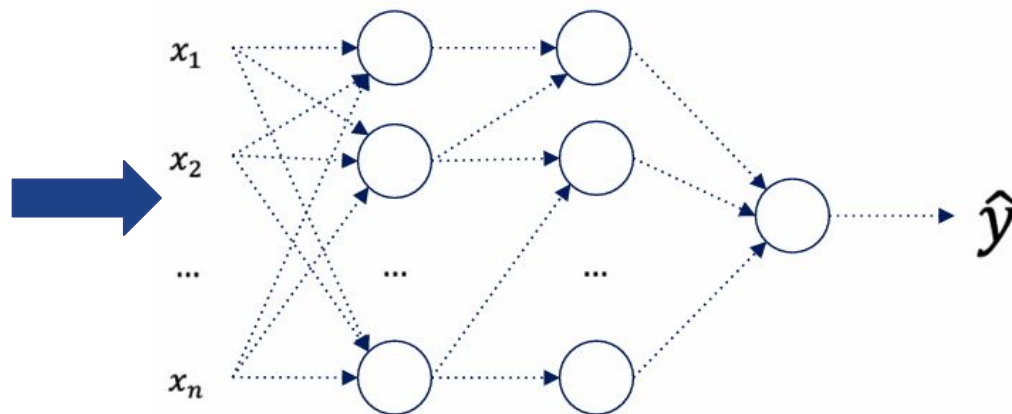
6 Millions

(2000 x 1000 x 3)

1 hidden layer

1000 neurons

The flattening concept



Input image dimension:

$2000 \times 1000 \times 3$

(Height $\times$ Width $\times$ Channels)

Input layer

6 Millions

($2000 \times 1000 \times 3$)



1 hidden layer

1000 neurons

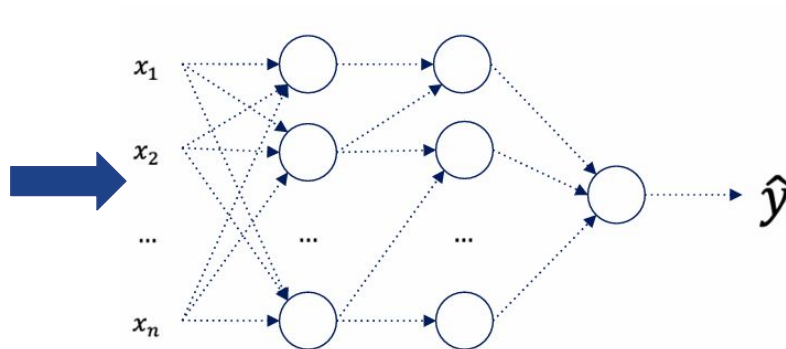


$6,000,000 \times 1000 =$

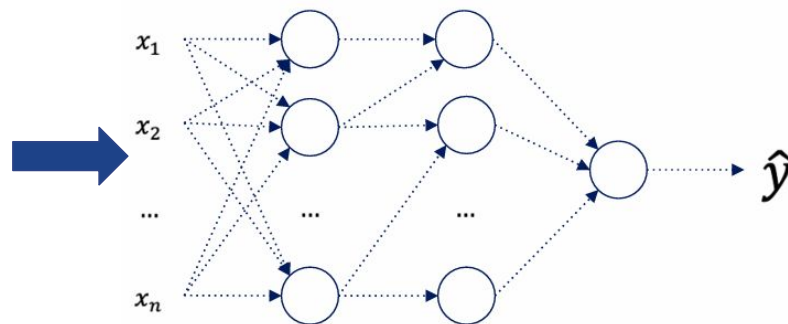
6,000,000,000

6 **BILLION** weights (+ 1000 biases)

Just for the first layer

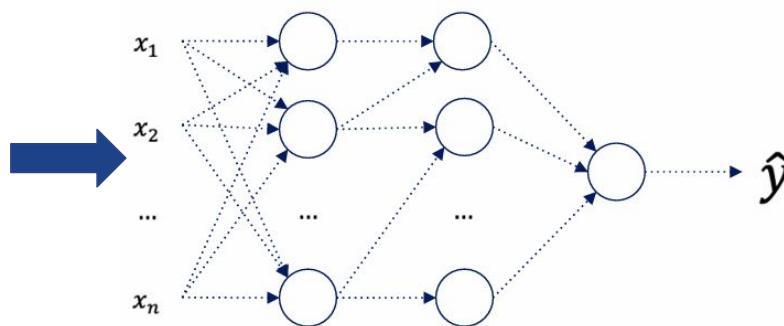


Storing **6B** parameters in float32
requires about **24 GB of memory.**

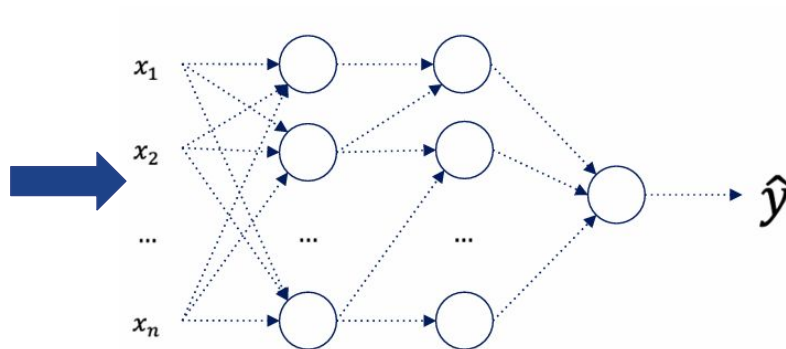


Fully connected neural networks are not adapted to image data

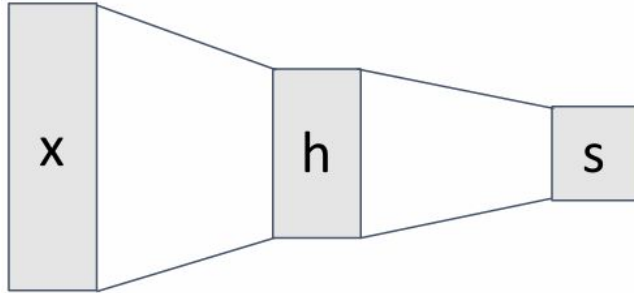
- Connect neurons in hidden layer to all neurons in input layer
- Too many parameters
- No spatial information



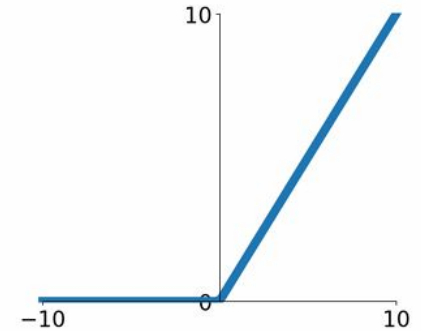
What if Convolutional Network can resolve this ?



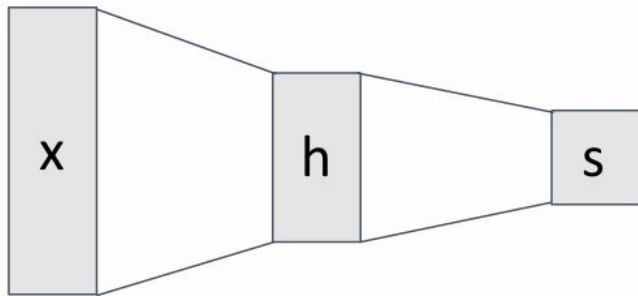
Fully-Connected Layers



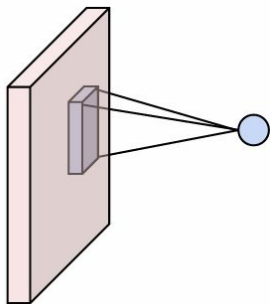
Activation Function



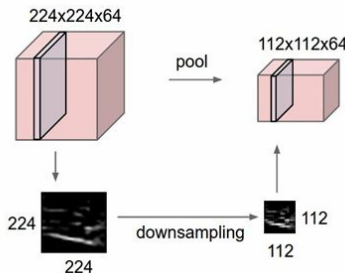
Fully-Connected Layers



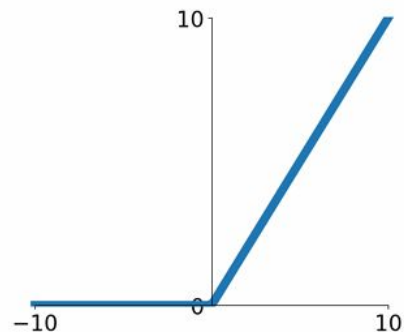
Convolution Layers



Pooling Layers



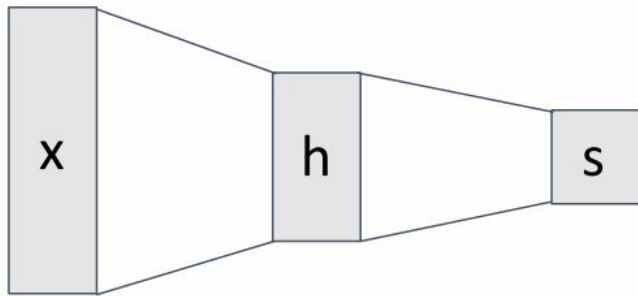
Activation Function



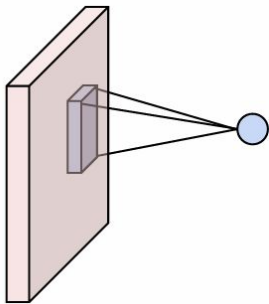
Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

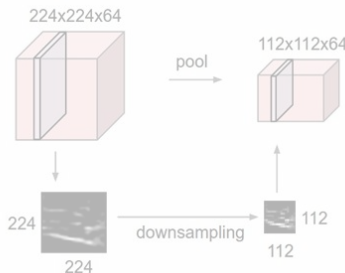
Fully-Connected Layers



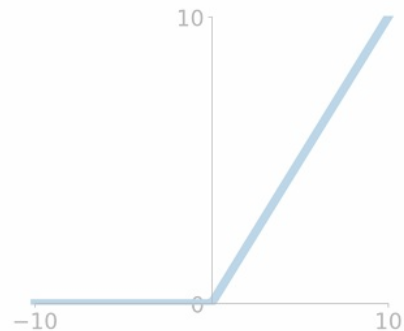
Convolution Layers



Pooling Layers



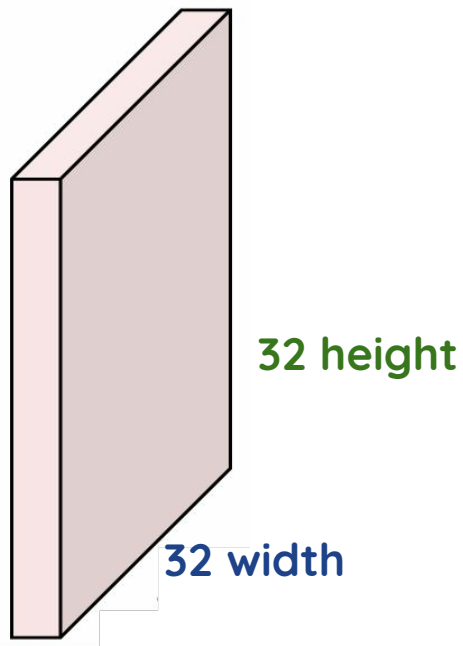
Activation Function



Normalization

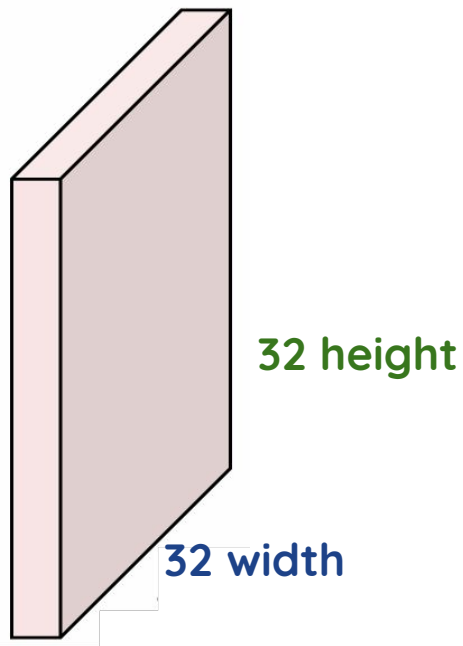
$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

3 x 32 x 32 image



We want to preserve **spatial structure** because nearby pixels are related

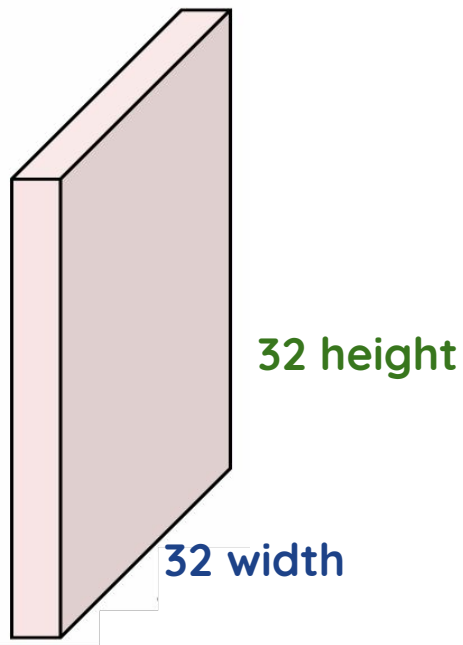
3 x 32 x 32 image



We want to preserve **spatial structure** because nearby pixels are related

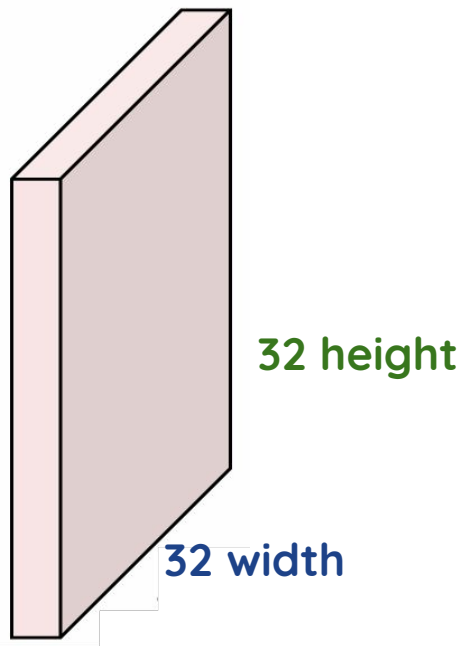
How do we compute features without **flattening**?

3 x 32 x 32 image



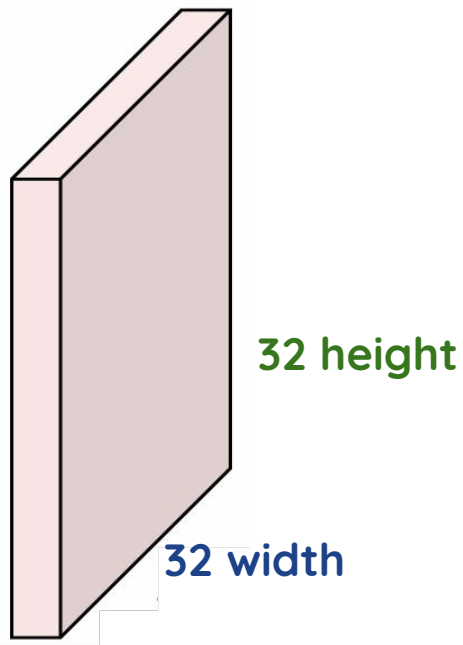
- Images are made of **local patterns**: edges, corners, textures (like we saw in the last chapter)
- We should look at **small neighborhoods**, not the whole image

3 x 32 x 32 image



To do that, we introduce a small learnable block called a **filter (kernel)**

3 x 32 x 32 image



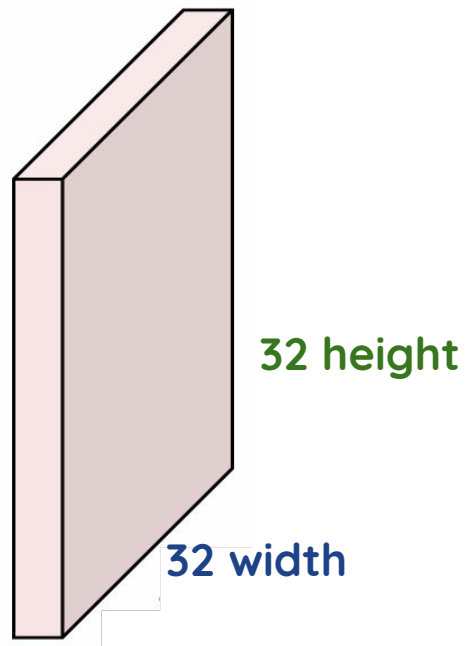
3 channels

A **filter** = small block of learnable **weights**

Example filter size: $3 \times 5 \times 5$



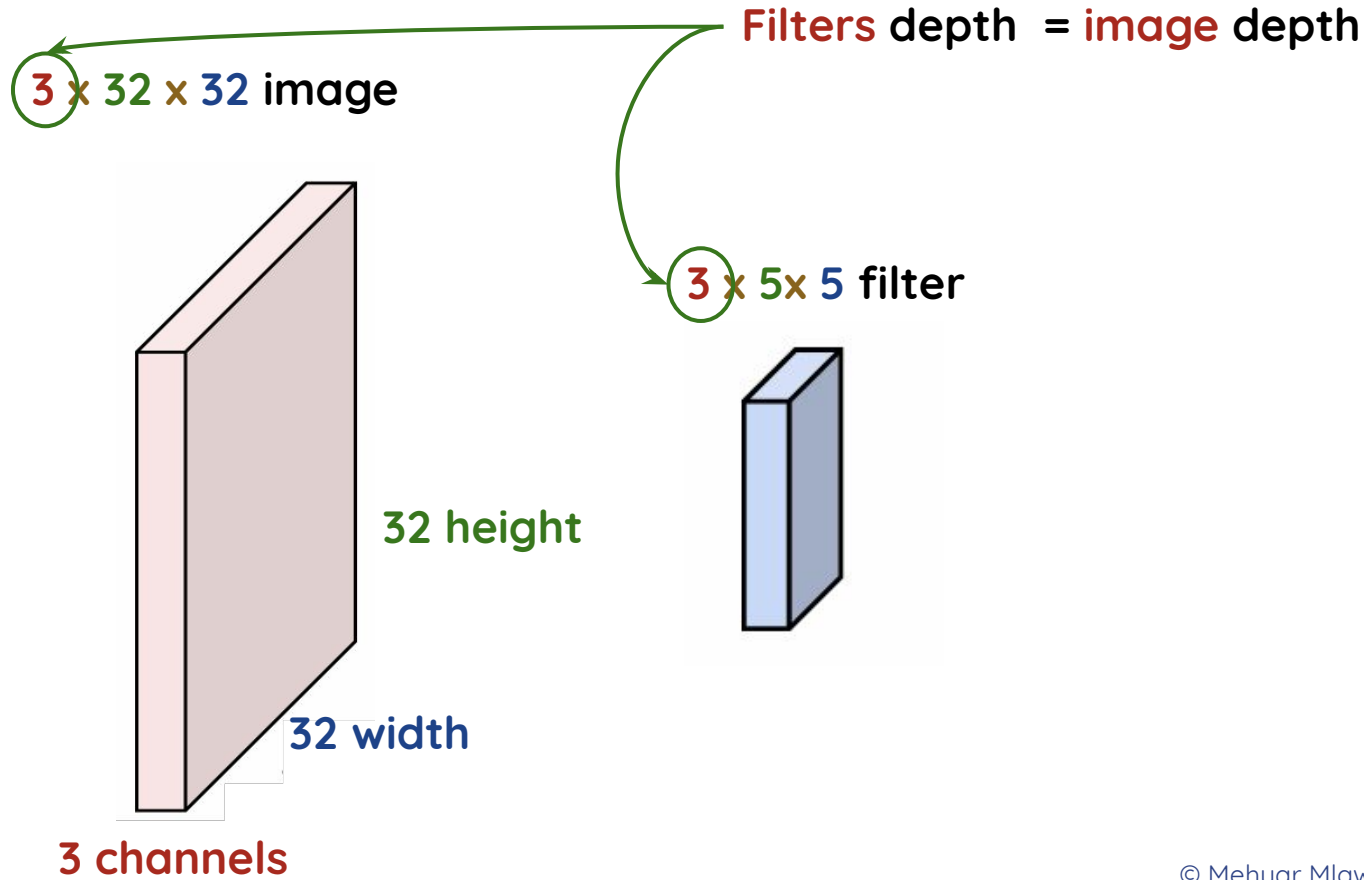
3 x 32 x 32 image

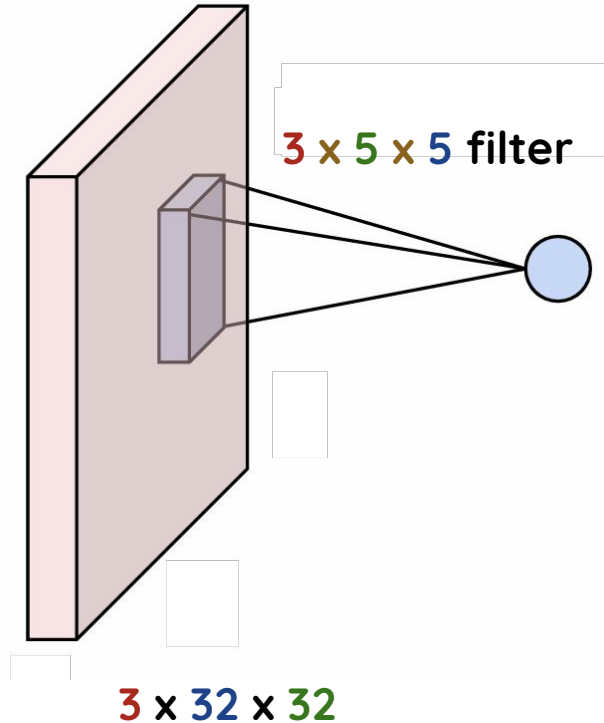


3 channels

3 x 5 x 5 filter

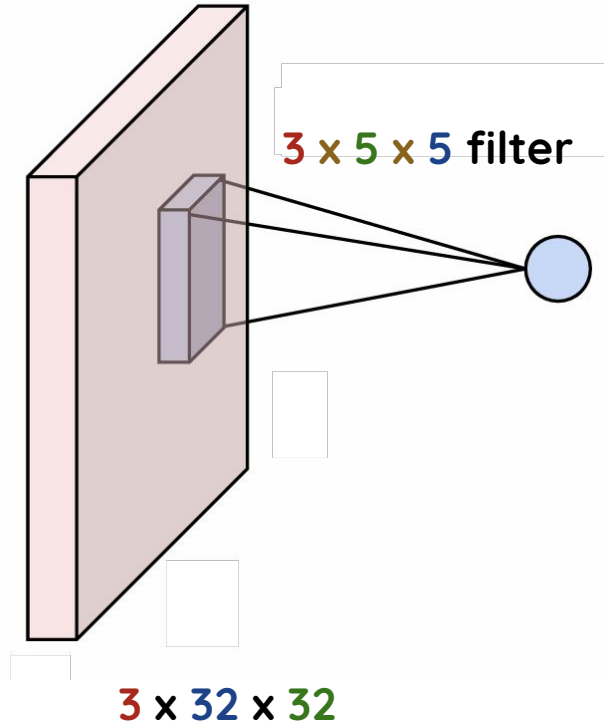






Produces **1 number**:

the result of taking a **dot product** between the filter and a small **$3 \times 5 \times 5$ chunk** of the image



1 Filter + 1 spatial position produces 1 number

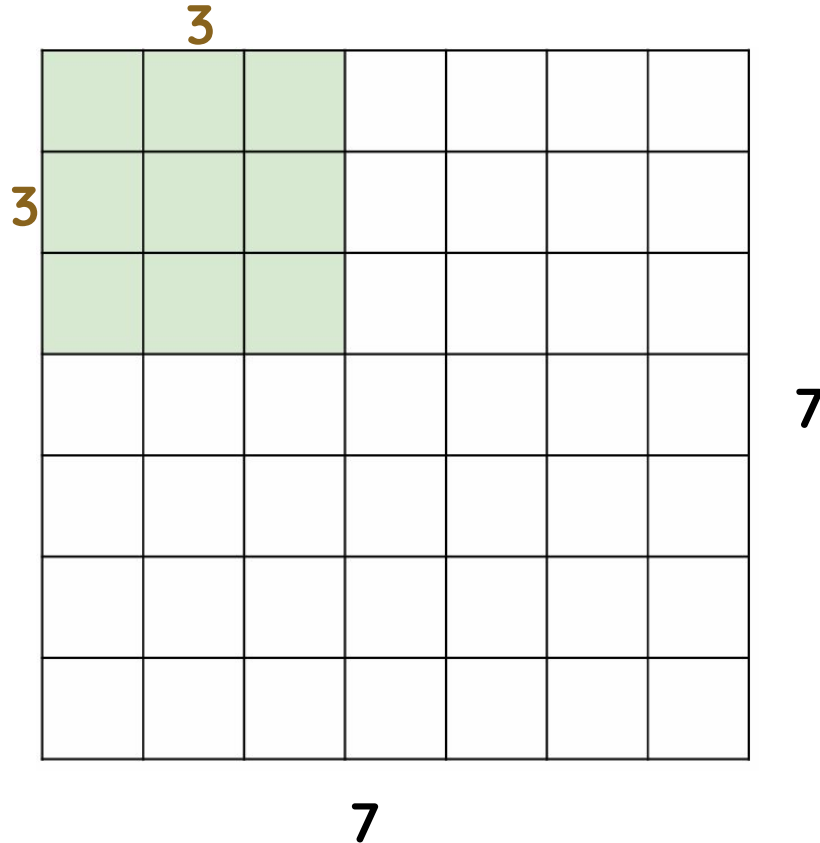
W: input

K: Filter (kernel)

$$\text{Output} = W - K + 1$$

$$= 32 - 5 + 1 = 28$$

A closer look!



$3 \times 3 \times 3$ filter

$3 \times 7 \times 7$ Image

$$\begin{aligned} &= 3 \cdot 1 + 0 \cdot 3 + 1 \cdot 2 \\ &\quad + 1 \cdot 3 + 5 \cdot 2 + 8 \cdot 1 \\ &\quad + 2 \cdot 2 + 7 \cdot 1 + 2 \cdot 1 \end{aligned}$$

3 x 3 x 3 filter

3 x 7 x 7 Image

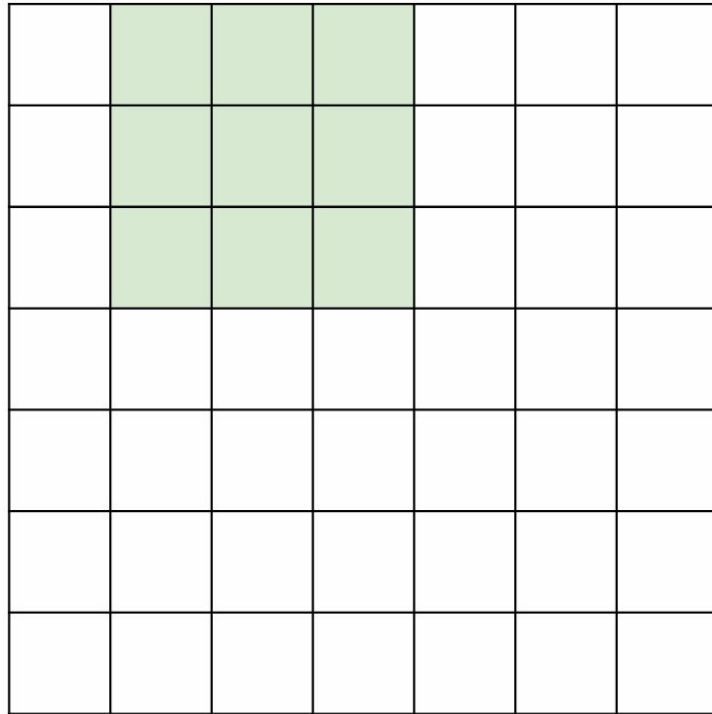
3	0	1	2	7	4	5
1	5	8	9	3	1	0
2	7	2	5	1	3	2
0	1	3	7	8	5	6
9	8	5	7	5	0	1
7	8	9	7	8	6	0
7	8	0	5	2	3	1

1	3	2
3	2	1
2	1	1

39				



Sliding the kernel



7

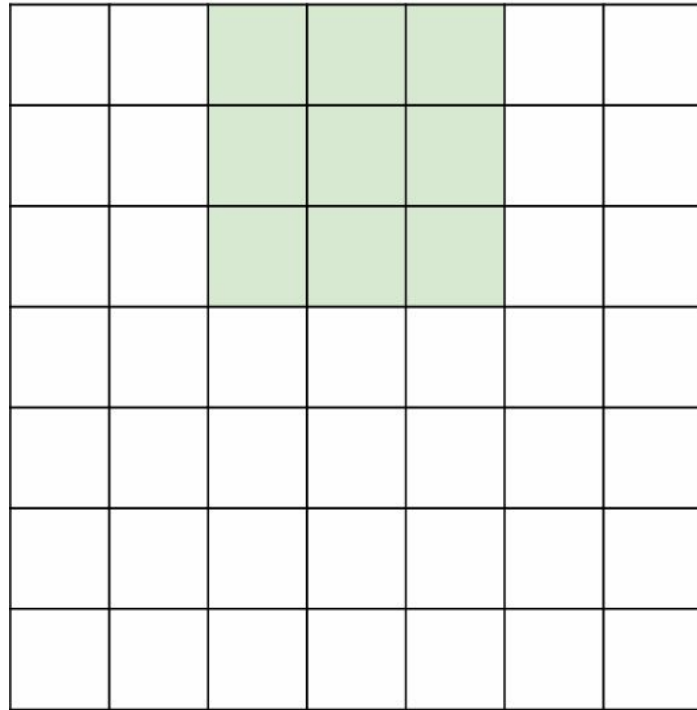
7

3 x 3 x 3 filter

3 x 7 x 7 Image



Sliding the kernel



7

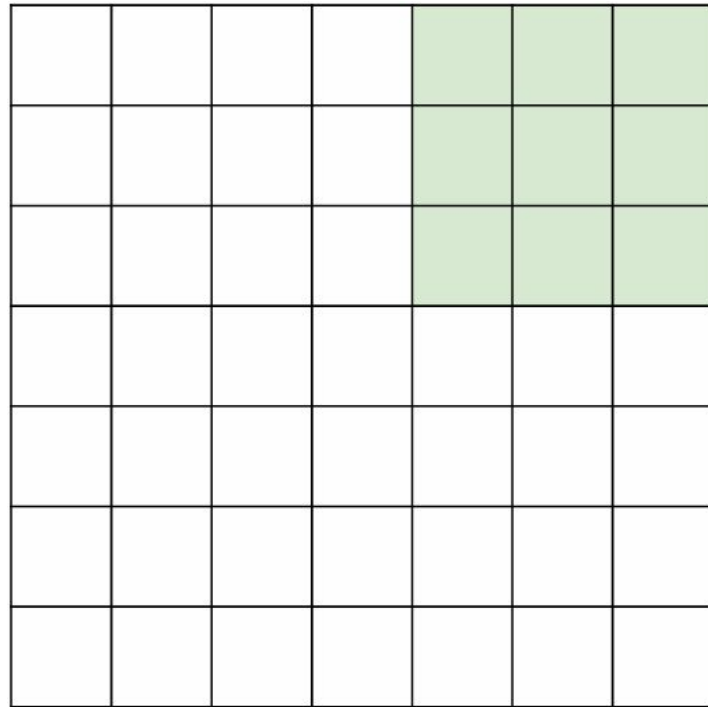
7

3 x 3 x 3 filter

3 x 7 x 7 Image

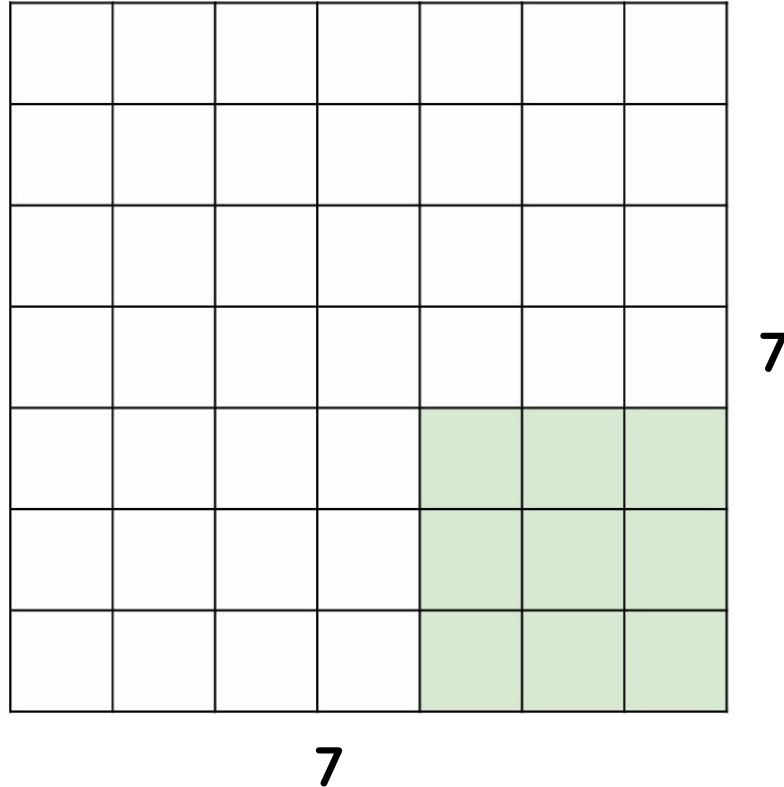


Sliding the kernel



3 x 3 x 3 filter

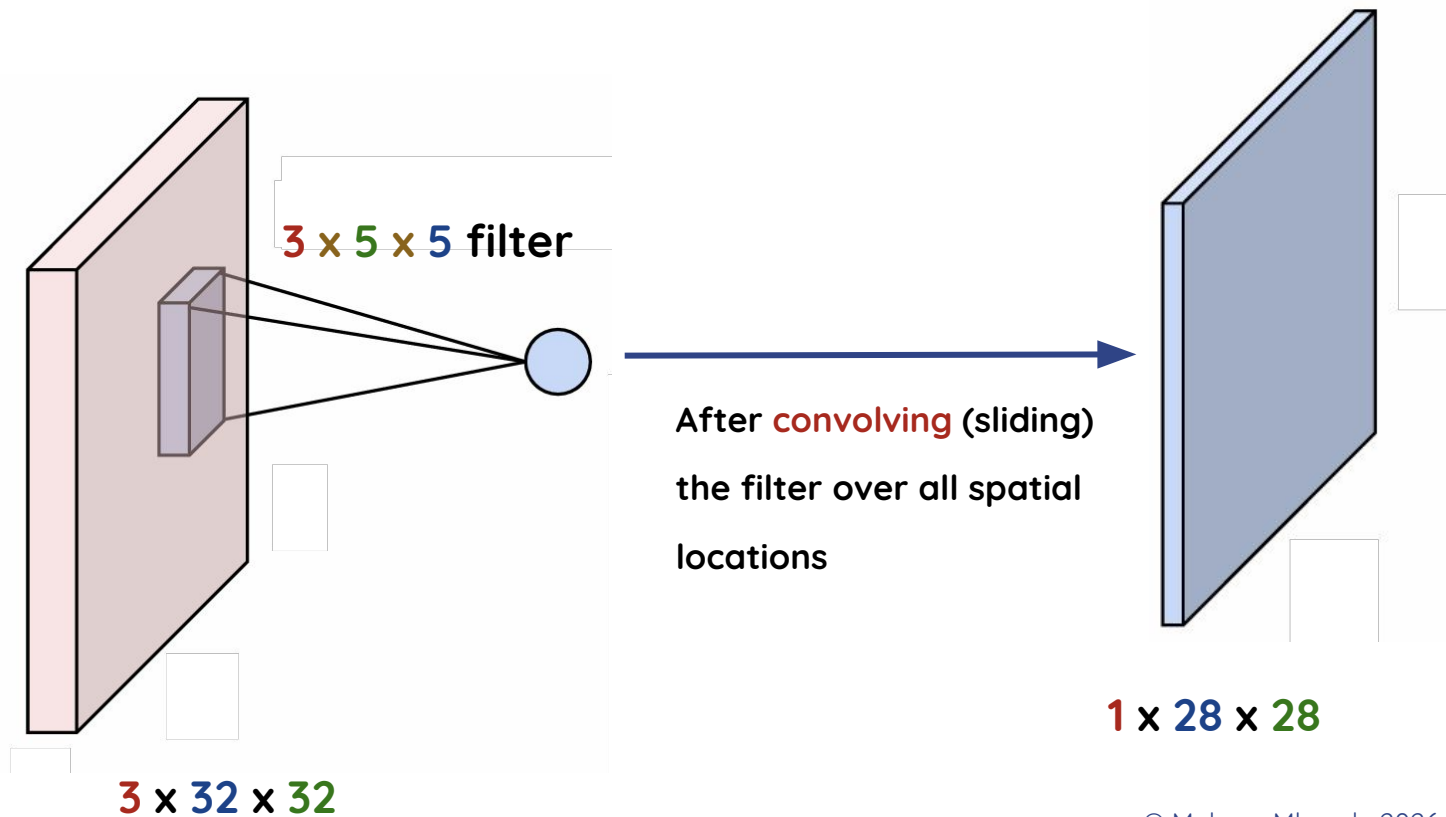
3 x 7 x 7 Image



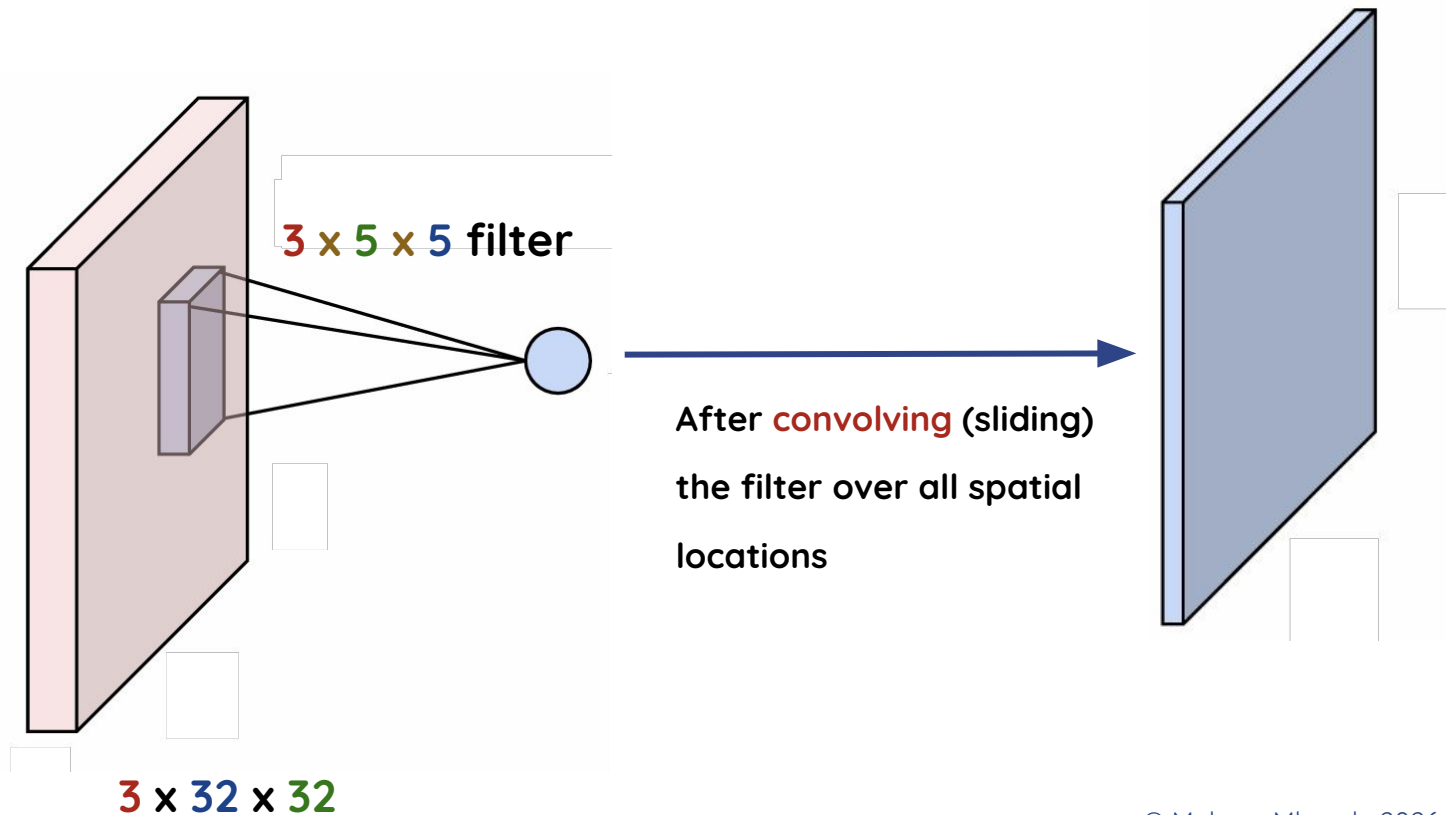
3 x 3 x 3 filter

3 x 7 x 7 Image

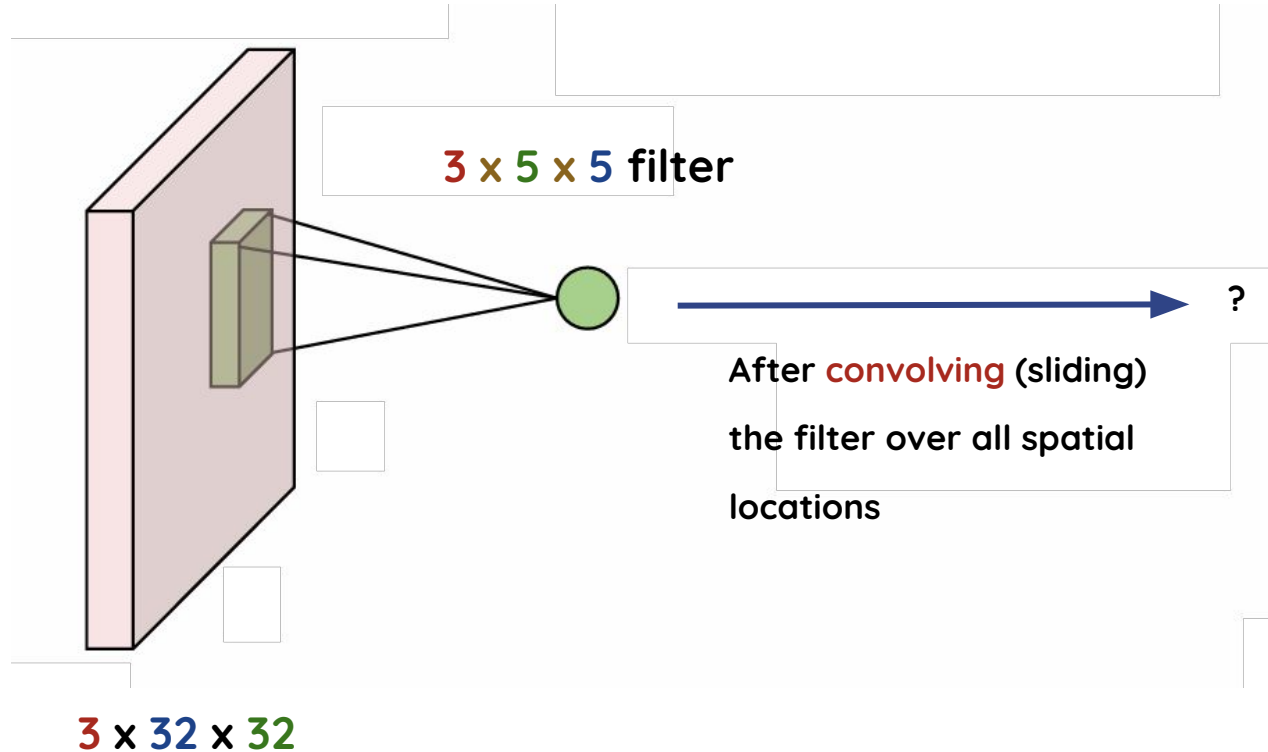
Output : 5 x 5



Activation map **1** x **28** x **28**

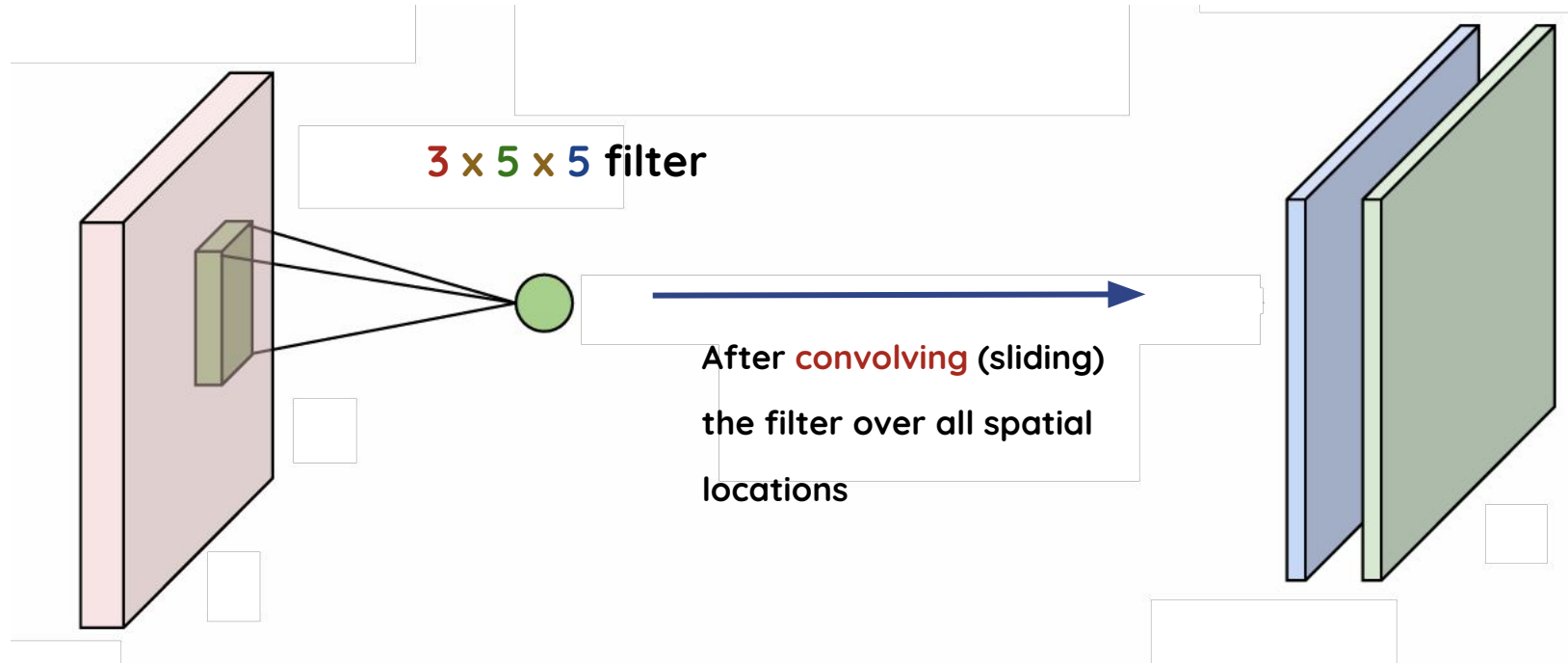


Consider repeating with a **second filter**

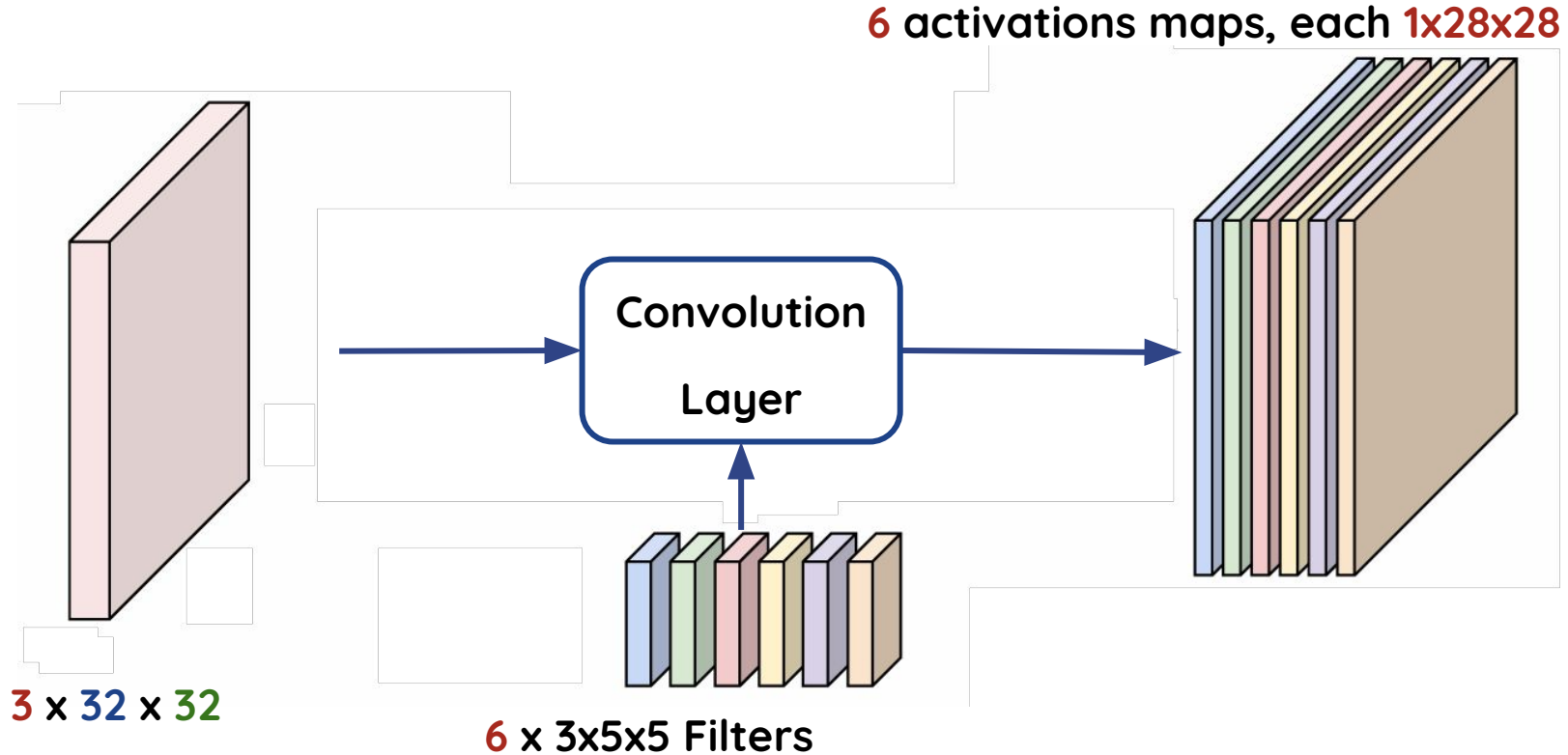


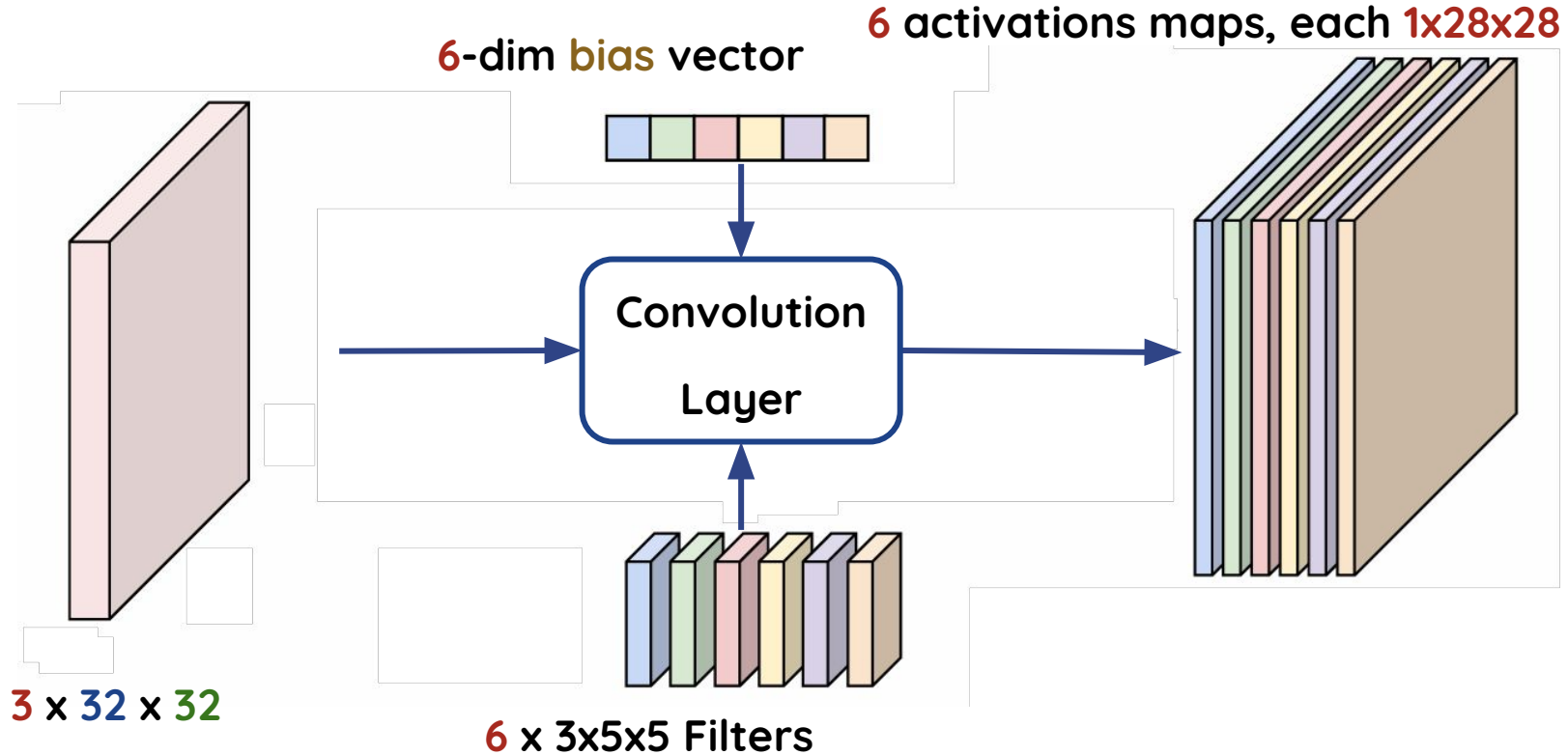
Consider repeating with a **second filter**

Two activation maps **1** x **28** x **28**



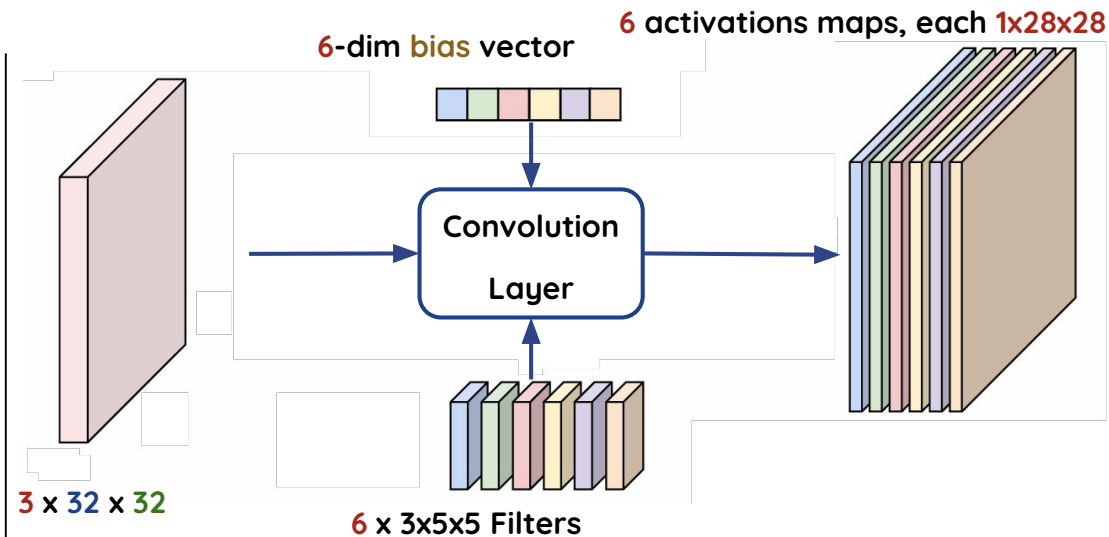
3 x **28** x **28**



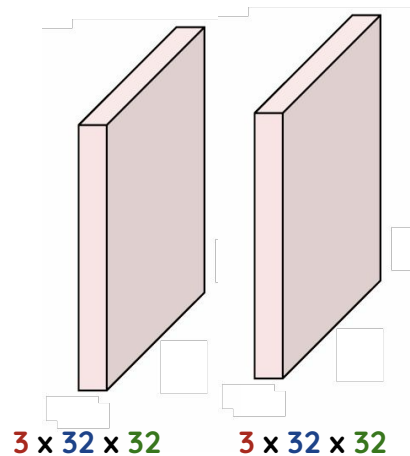


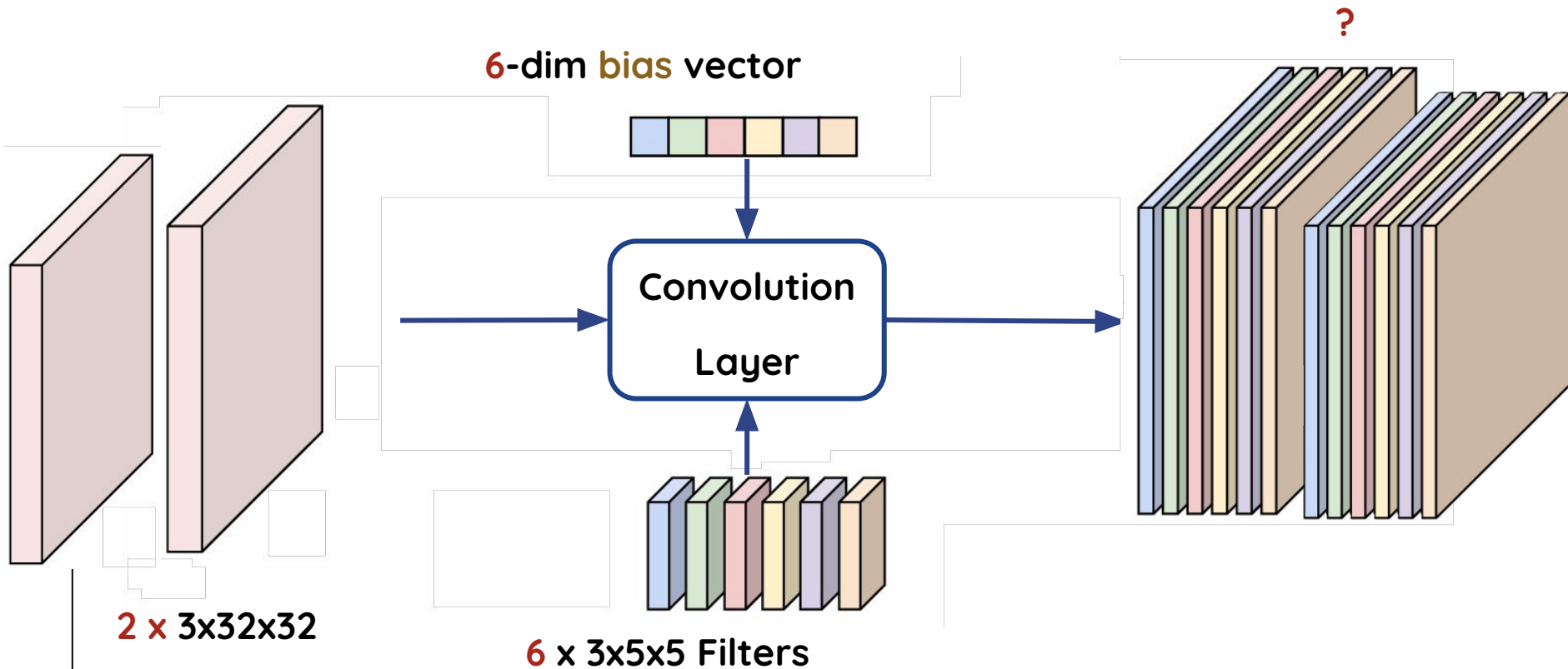
For each filter k :

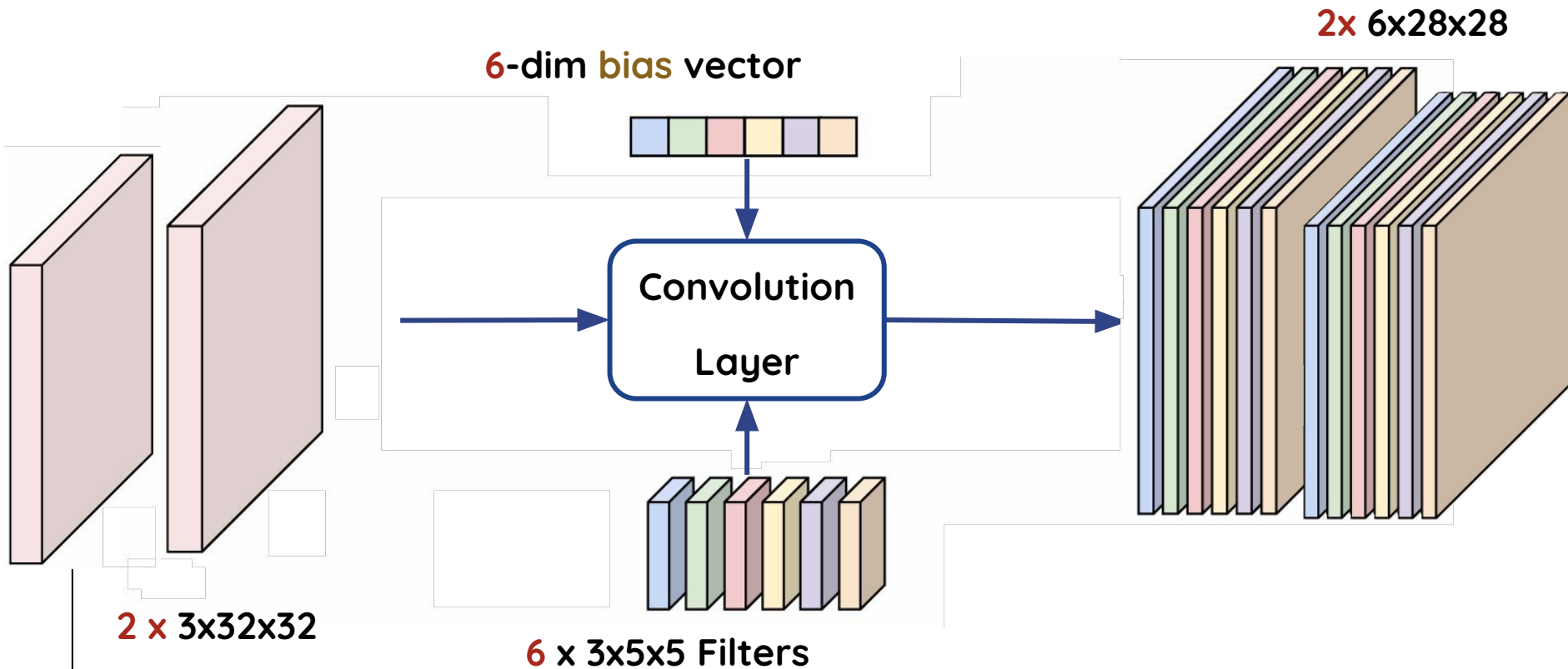
$$z_{i,j}^{(k)} = (W_k * X)_{i,j} + b_k$$



What if we have batch of images ?







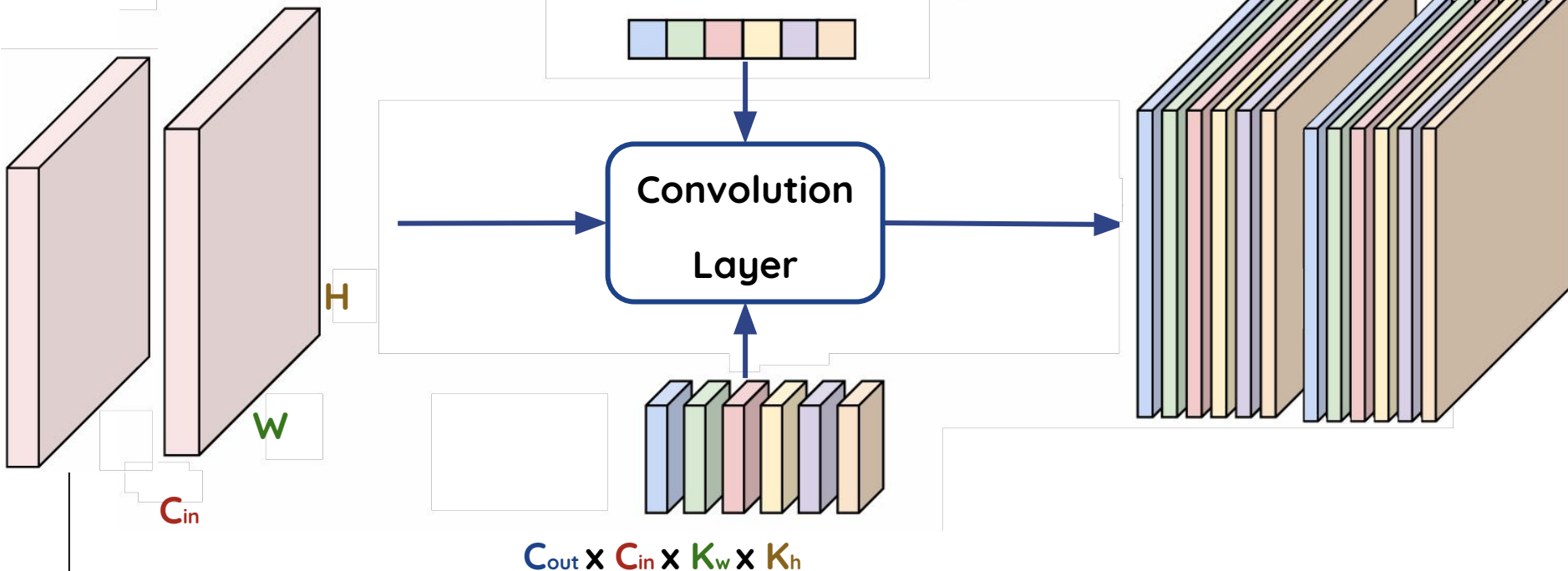
Convolution Layer

General case

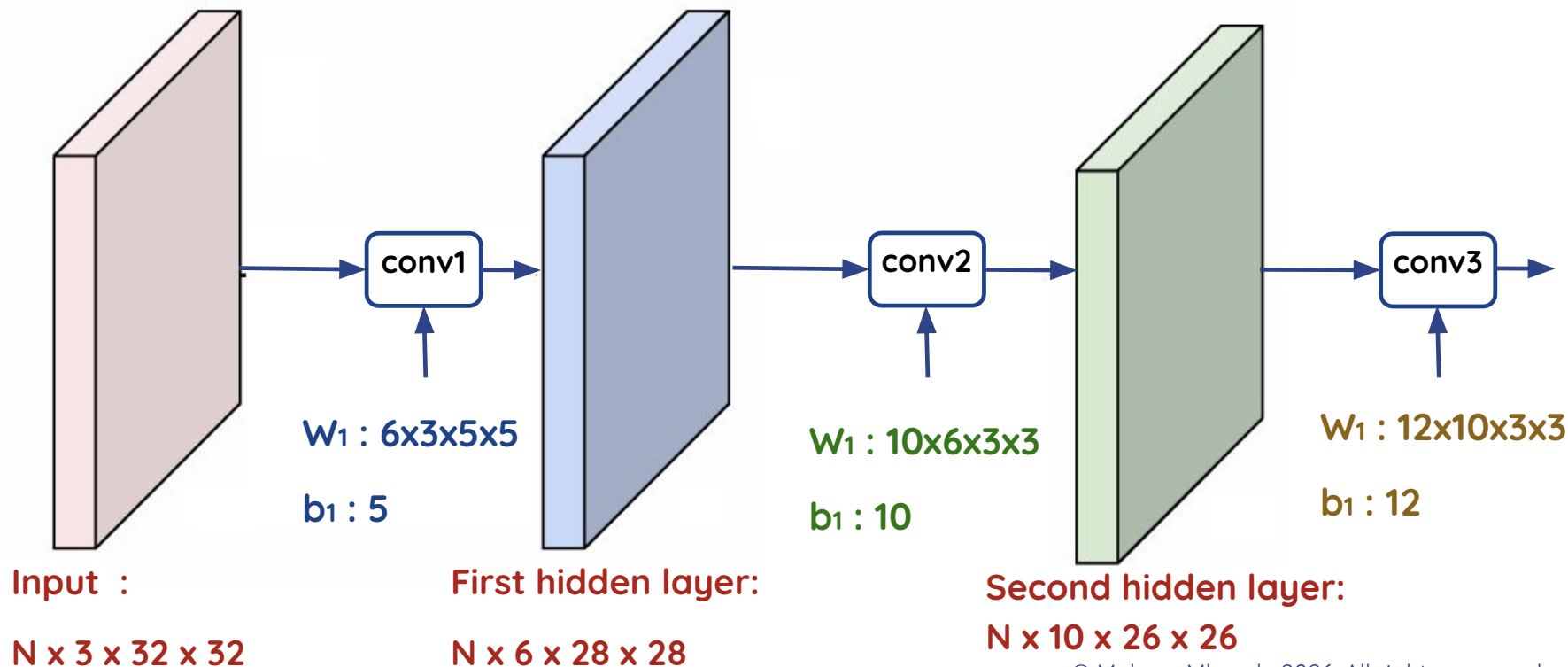
$N \times C_{in} \times H \times W$

C_{out} -dim bias vector

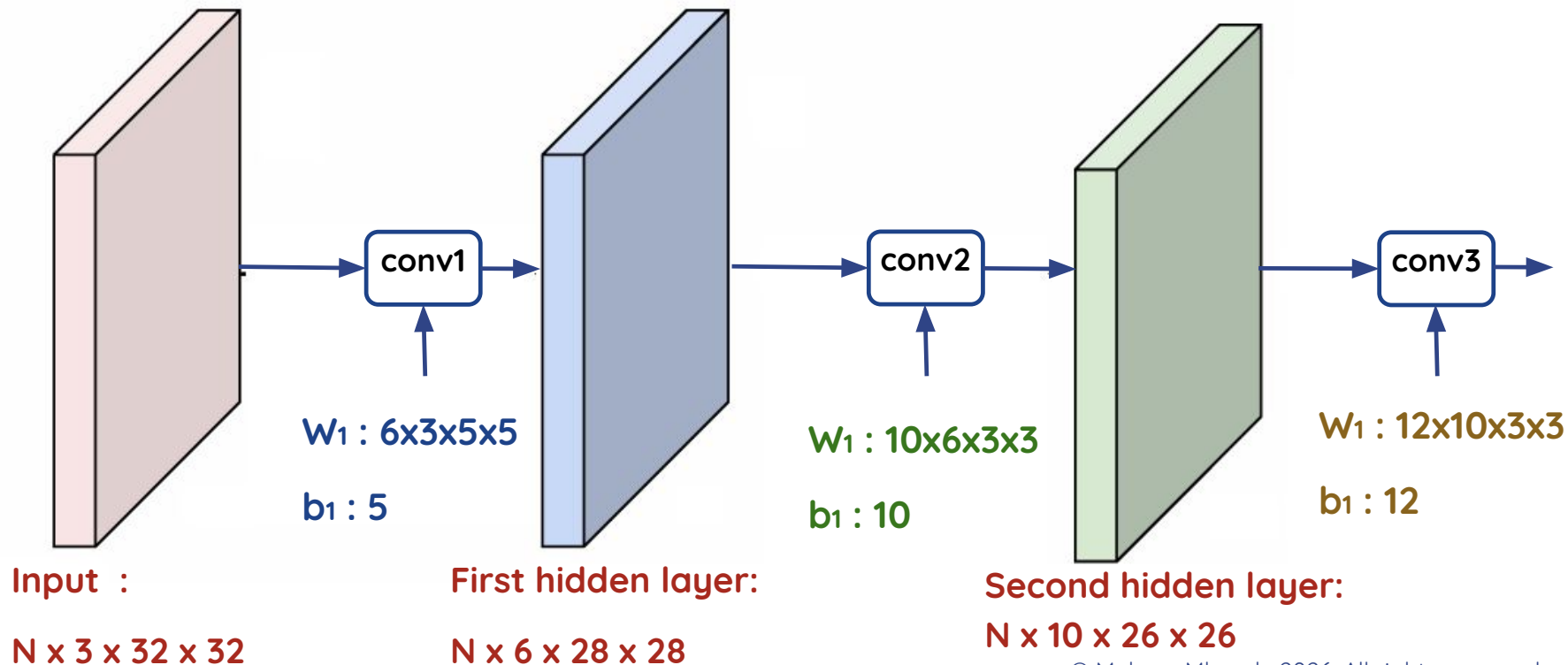
$N \times C_{out} \times H' \times W'$



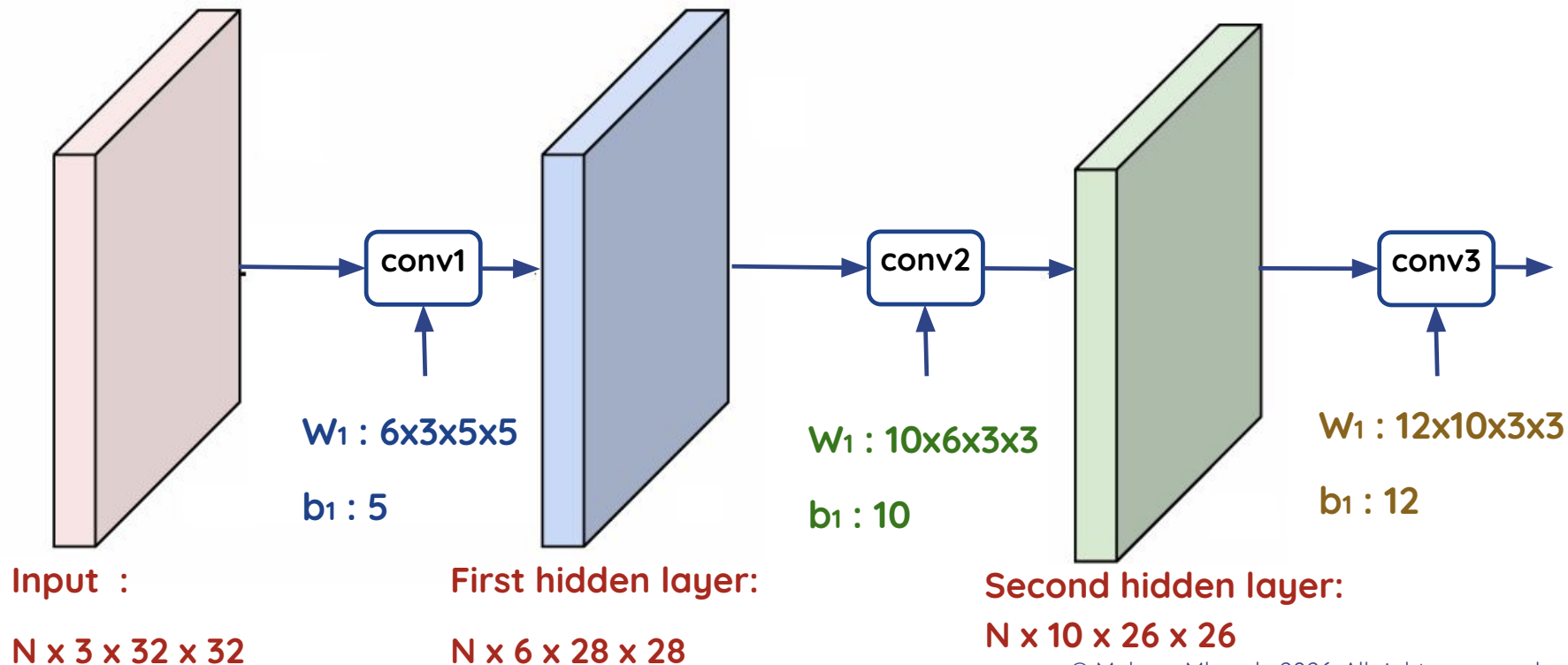
filters



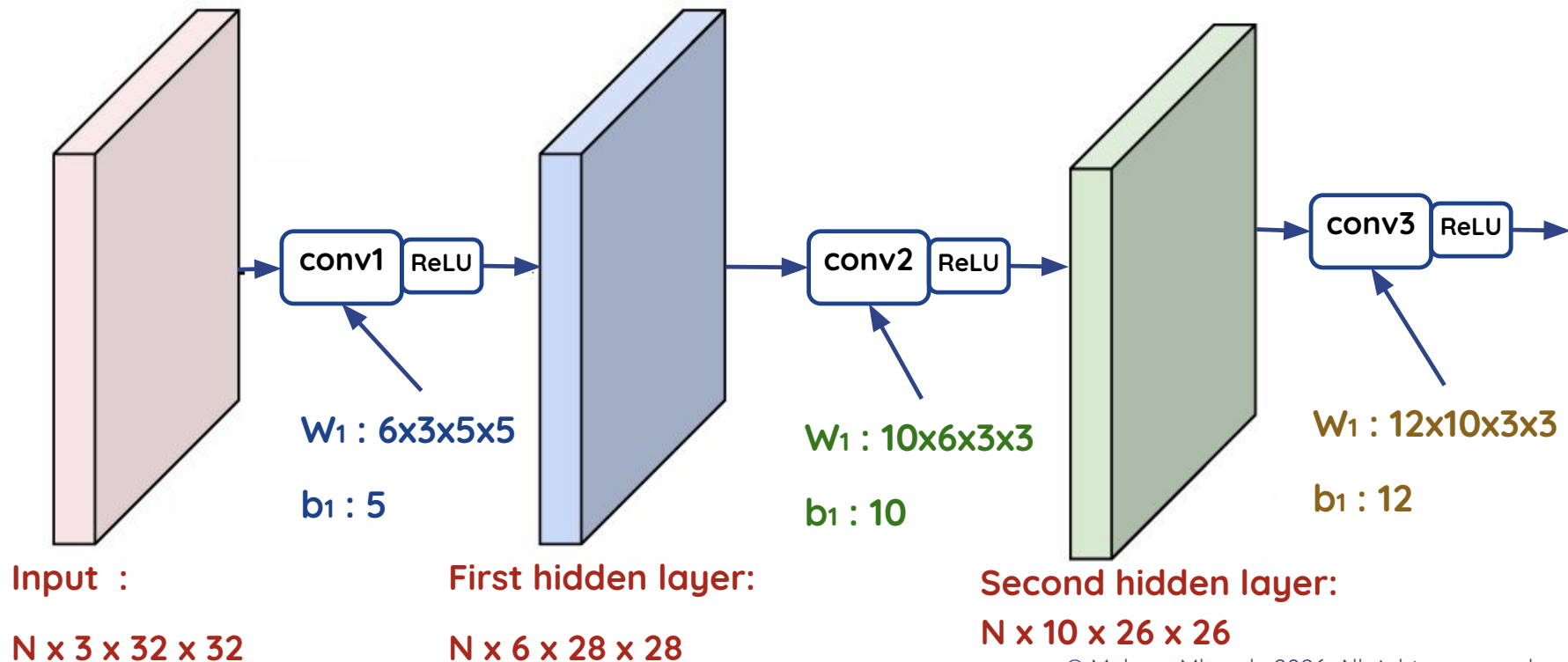
So convolution is just linear **filtering**



We are missing **non-linearity**



Let's try to add some **non-linearity**



Let's try to add some **non-linearity**

Conv → ReLU → Conv → ReLU → Conv → ReLU

Let's try to add some **non-linearity**

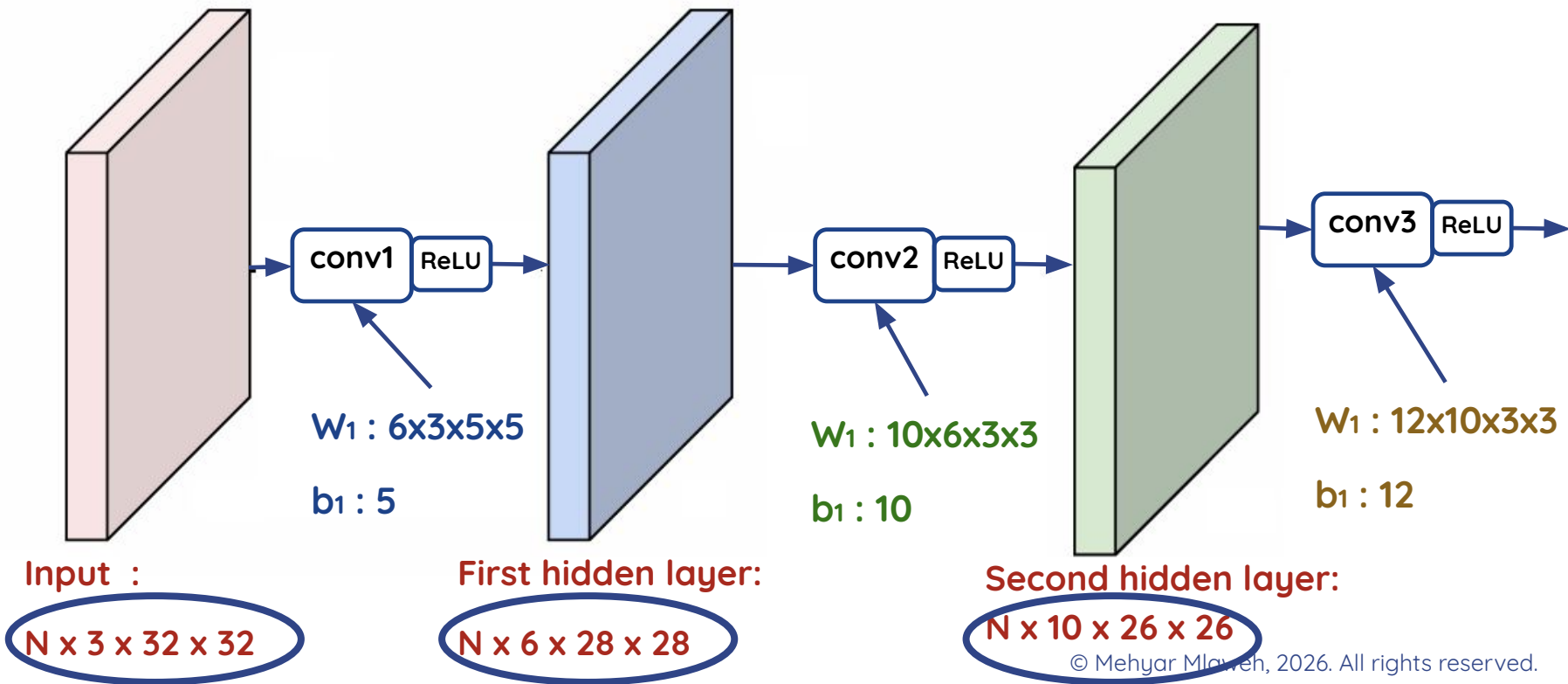
Conv → ReLU → Conv → ReLU → Conv → ReLU

$$\text{ReLU}(x) = \max(0, x)$$

Input: 32×32

After conv1: 28×28

After conv2: 26×26

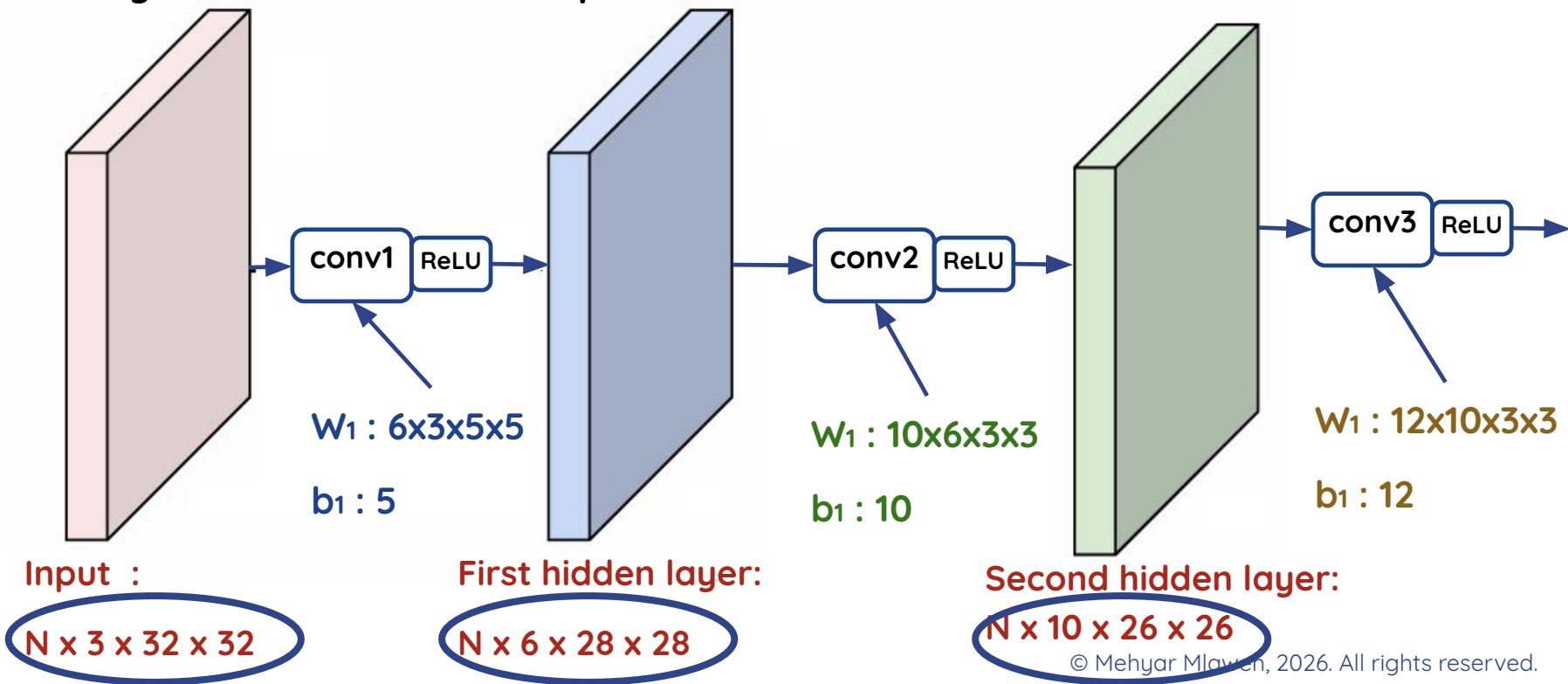


Input: 32×32

After conv1: 28×28

After conv2: 26×26

Every convolution **reduces** spatial resolution



$32 \rightarrow 28 \rightarrow 26 \rightarrow 24 \rightarrow \dots$

- Deep networks **shrink** too fast
- We may **lose** spatial information

$32 \rightarrow 28 \rightarrow 26 \rightarrow 24 \rightarrow \dots$

- Deep networks **shrink** too fast
- We may **lose** spatial information

Solution : padding

Add **zeros** around the input

$32 \rightarrow 28 \rightarrow 26 \rightarrow 24 \rightarrow \dots$

- Deep networks **shrink** too fast
- We may **lose** spatial information

Solution : padding

Add **zeros** around the input

Solution : padding

Add **zeros** around the input

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Solution : padding

Add **zeros** around the input

Input : W

Filter : K

Padding : P

Output : $W - K + 1 + 2.P$

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Solution : padding

Add **zeros** around the input

Input : W

Filter : K

Padding : P

Output : $W - K + 1 + 2.P$

Very common :

Set $P = (K - 1) / 2$

To make output have size as input!

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Example

Input : $3 \times 32 \times 32$

Filter : $3 \times 5 \times 5$

Output with $P = 0$:

$3 \times 28 \times 28$

Output with $P = 2$:

$3 \times 32 \times 32$

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

But doesn't adding **zeros**
introduce **noise** and
corrupt the image?

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

But doesn't adding **zeros** introduce **noise** and corrupt the image?

No!

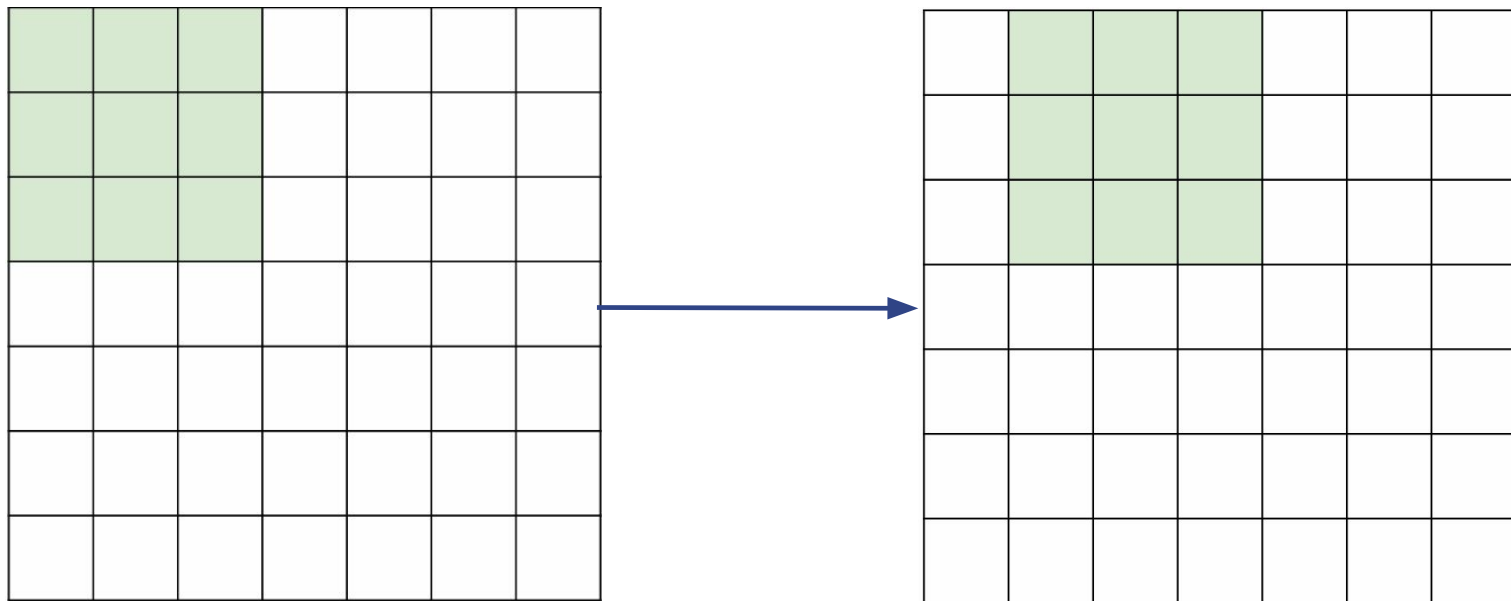
1 - All real pixels remain unchanged.

2 - It prevents losing border information

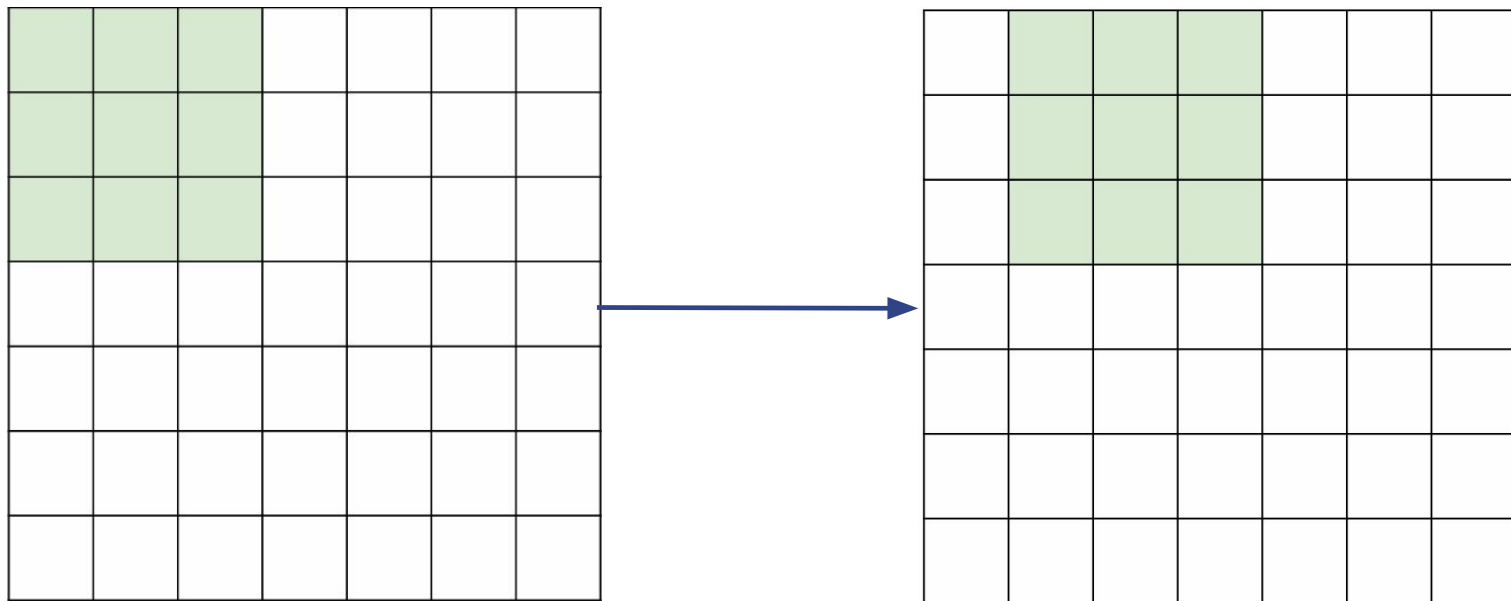
0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

So far, our filter moves **one pixel** at a time.

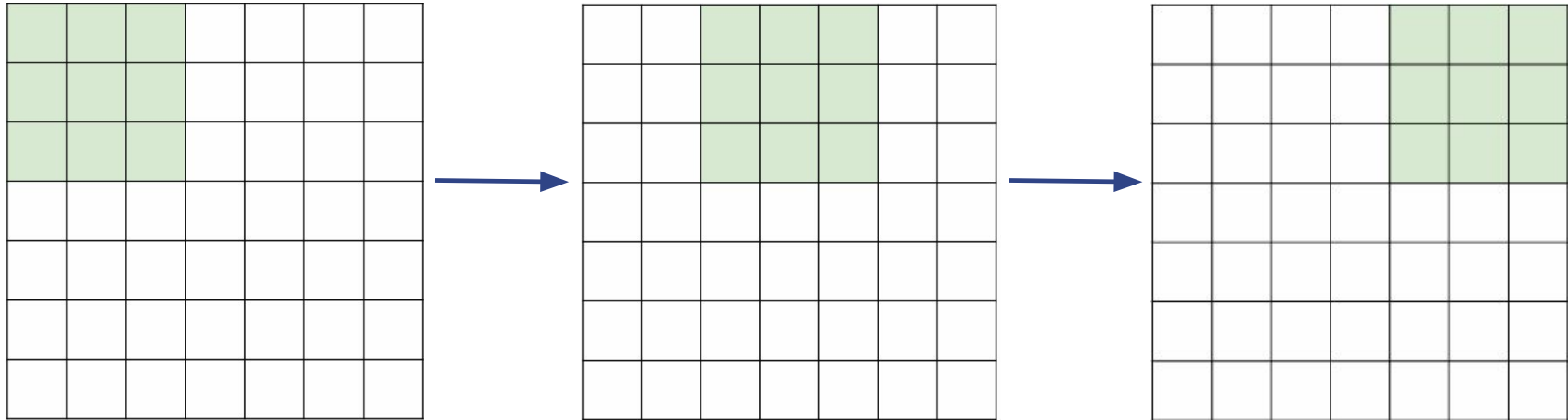
But do we really need to compute every position?



Stride = 1 $\rightarrow$ move 1 pixel (default)



Stride = 2 $\rightarrow$ move 2 pixels (skip one position)



General Formula

$$W_{out} = \frac{W - K + 2P}{S} + 1$$

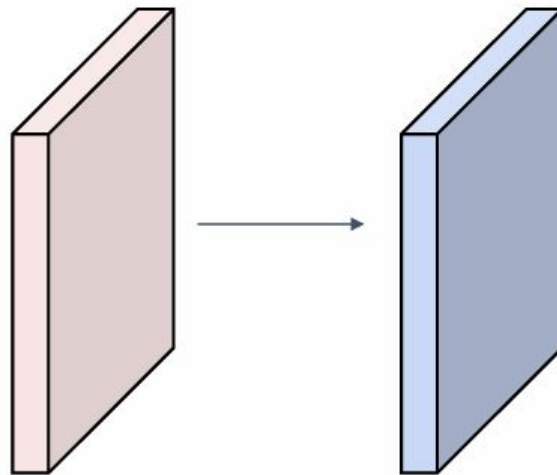
Convolution Example

$$W_{out} = \frac{W - K + 2P}{S} + 1$$

What If we have input 3 x 32 x 32

10 3x5x5 filters with stride 1 and pad 1

Output volume size: ?



Convolution Example

$$W_{out} = \frac{W - K + 2P}{S} + 1$$

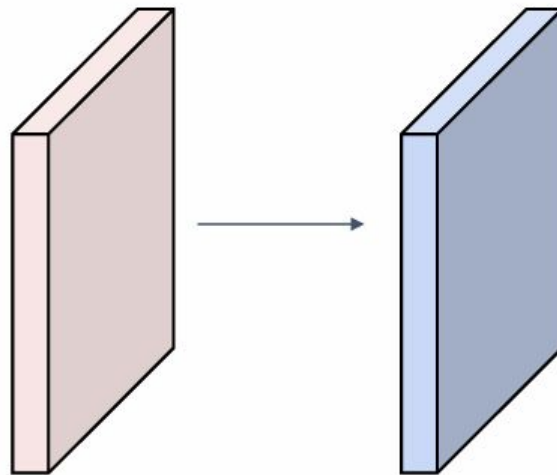
What If we have input 3 x 32 x 32

10 3x5x5 filters with stride 1 and pad 2

Output volume size:

$(32 + 2 \cdot 2 - 5) / 1 + 1 = 32$ spatially,

so 10 x 32 x 32



Convolution Example

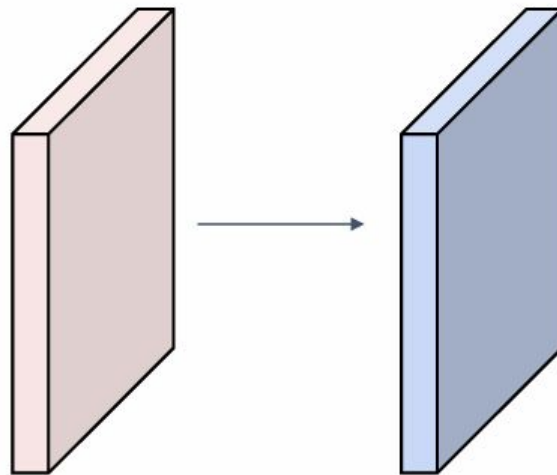
$$W_{out} = \frac{W - K + 2P}{S} + 1$$

What If we have input 3 x 32 x 32

10 3x5x5 filters with stride 1 and pad 2

Output volume size: 10 x 32 x 32

Number of learnable parameters: ?



Convolution Example

$$W_{out} = \frac{W - K + 2P}{S} + 1$$

What If we have input **3** x 32 x 32

10 **3x5x5** filters with stride 1 and pad 2

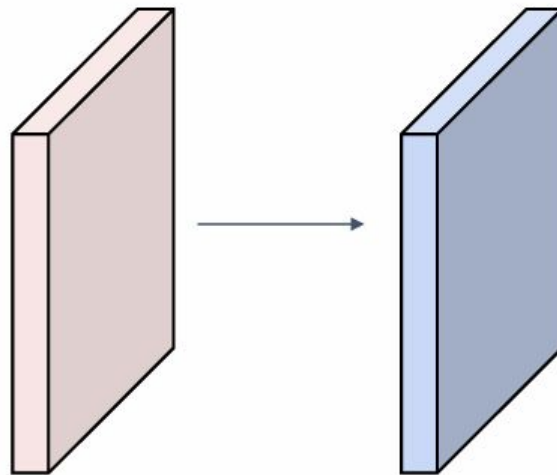
Output volume size: 10 x 32 x 32

Number of learnable parameters:

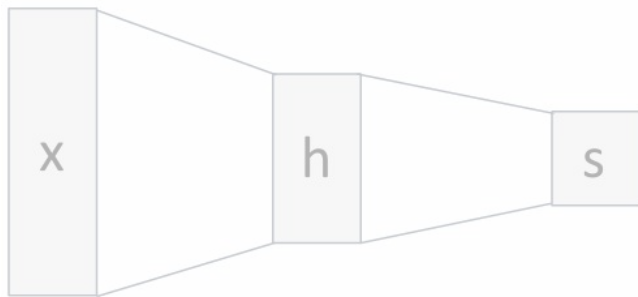
Parameters per filter : **3** * **5** * **5** +1 (bias)

= **76**

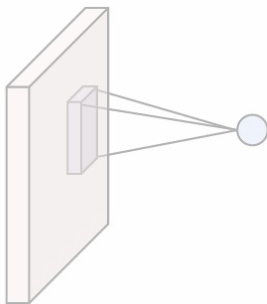
10 Filters so total is **760**



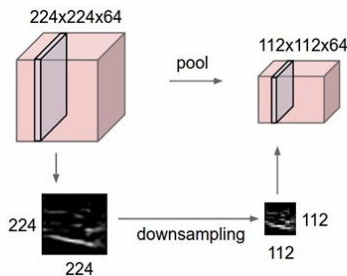
Fully-Connected Layers



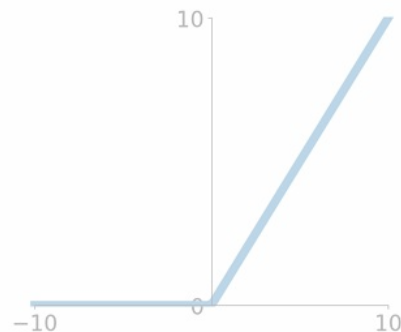
Convolution Layers



Pooling Layers



Activation Function



Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

After several convolution layers, what happens to spatial size?

After several convolution layers, what happens to spatial size?

Even with padding, feature maps remain large:

Example:

$10 \times 32 \times 32$

→ next layer still 32×32

After several convolution layers, what happens to spatial size?

Even with padding, feature maps remain large:

Example:

$10 \times 32 \times 32$

→ next layer still 32×32

This is computationally **expensive**

Solution : Pooling layers

Pooling is a **down-sampling** operation that reduces the **spatial dimensions** (width and height) of the input **feature map** while retaining important features.

The pooling operator is typically applied **after** a convolutional layer

Pooling operation **does not** have **learnable parameters**

Different parameters define the pooling operation :

- Filter size
- Stride parameter
- Pooling operation : Max Pooling, Average Pooling, Min Pooling, etc

Example of Max and Average Pooling

3	0	7	4
1	5	3	1
4	2	2	8
2	4	3	9

$\Rightarrow$

5	7
4	9

Max Pooling

3	0	7	4
1	5	3	1
4	2	2	8
2	4	3	9

$\Rightarrow$

2.25	3.75
3	5.5

Average Pooling

The main advantage of pooling is :

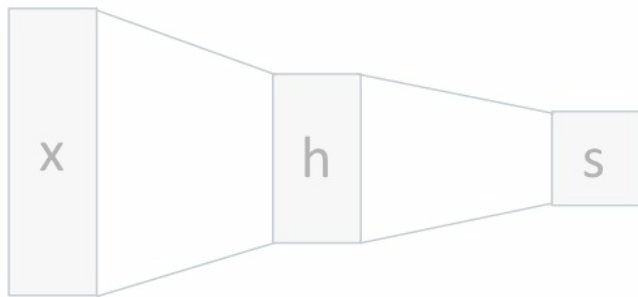
- Reduces the number of parameters in the network
- which helps prevent overfitting and improves the computational efficiency of the model

Pooling also helps reducing noise Especially
Max Pooling that only extract **higher**
activation

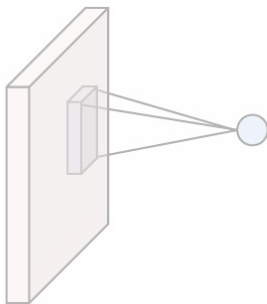


[source : medium.com/@bdhuma/]

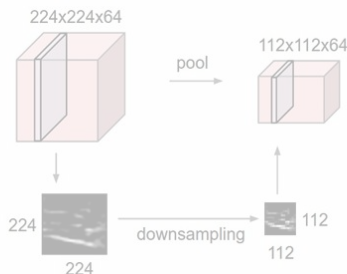
Fully-Connected Layers



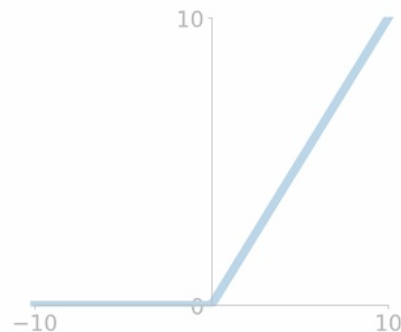
Convolution Layers



Pooling Layers



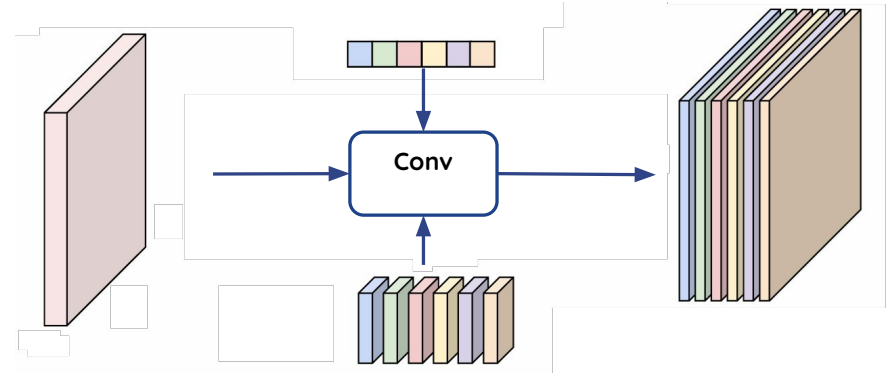
Activation Function



Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

After convolution and ReLU, what are the values inside the feature maps?

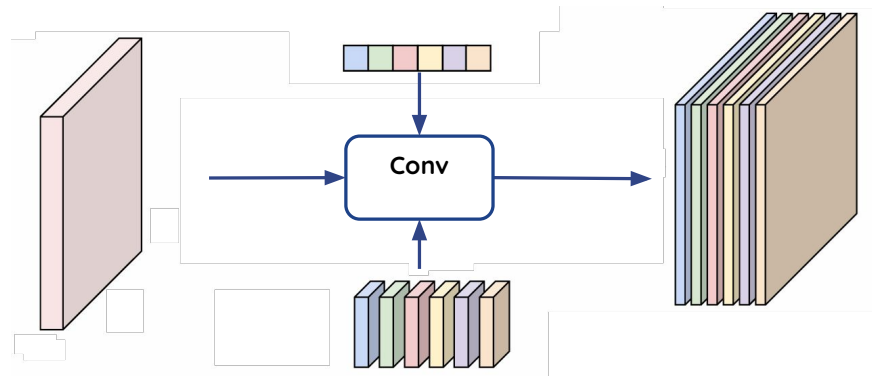


After convolution and ReLU, what are the values inside the feature maps?

They can be:

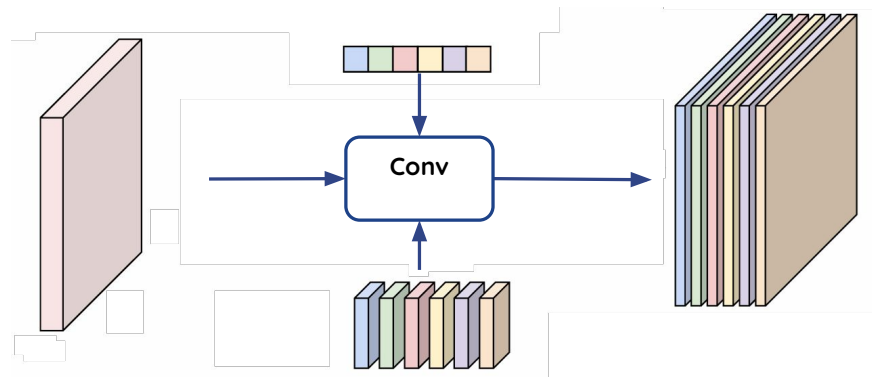
- Very large
- Very small
- Different distributions across channels
- Different distributions across batches

Training can become **unstable**.



Idea: “**Normalize**” the outputs of a layer so they have zero mean and **unit variance** ?

Improves optimization.



In CNNs we usually use **Batch Normalization**

For each channel:

$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}] + \varepsilon}}$$

Then:

$$y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}$$

$\mu^{(k)}$: mean of channel k over the mini-batch

$\sigma^2^{(k)}$: variance of channel k

ε : small constant for numerical stability

$\gamma^{(k)}$: learnable scale parameter

$\beta^{(k)}$: learnable shift parameter

Usually applied **after convolution** and
before activations.

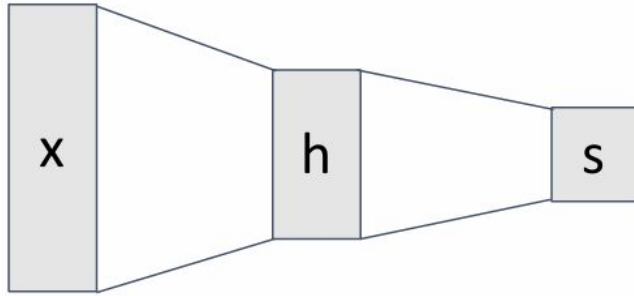
$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}] + \varepsilon}}$$

Usually applied **after convolution** and **before activations**.

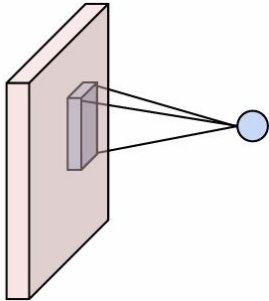
Because we want to normalize the linear activations before applying the non-linearity.

$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}] + \varepsilon}}$$

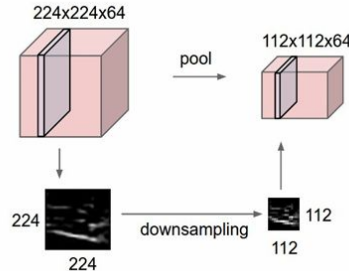
Fully-Connected Layers



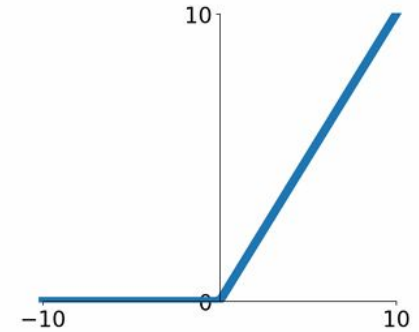
Convolution Layers



Pooling Layers



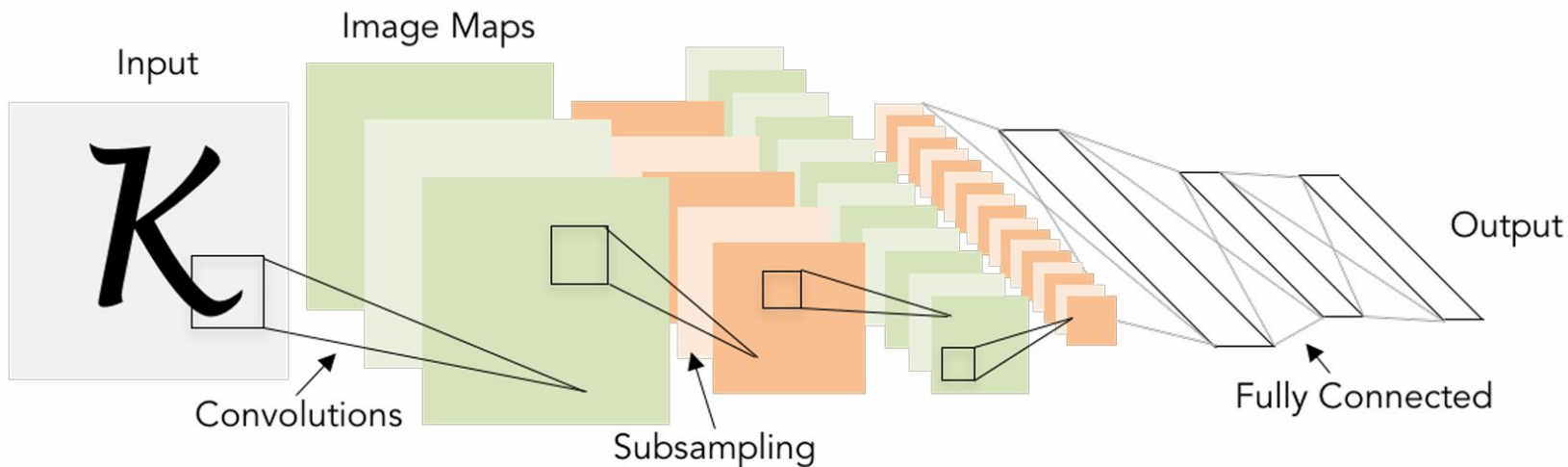
Activation Function



Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Problem : What is the right way to combine all these components ?



CNNs Architectures

Books

- Computer Vision: Models, Learning, and Inference (2012). <https://amzn.to/2rxrdOF>
- Programming Computer Vision with Python (2012). <https://amzn.to/2QKTaAL>
- Multiple View Geometry in Computer Vision (2004). <https://amzn.to/2LfHLE8>
- Computer Vision: Algorithms and Applications (2010). <https://amzn.to/2Lclt4J>
- Computer Vision: A Modern Approach (2002). <https://amzn.to/2rv7AqJ>
- Deep Learning for Computer Vision : Image Classification, Object Detection and Face Recognition in Python(2019). <https://machinelearningmastery.com/deep-learning-for-computer-vision/>
- Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow — Aurélien Géron (2019)
<https://amzn.to/3uZbN6Z>

Articles & Online Resources

- Krizhevsky, Sutskever, Hinton (2012) — ImageNet Classification with Deep CNNs (AlexNet — deep learning breakthrough)
- Improved Texture Networks: Maximizing Quality and Diversity in Feed-forward Stylization and Texture Synthesis, 2017
- “Computer vision,” Wikipedia. https://en.wikipedia.org/wiki/Computer_vision
- “Machine vision,” Wikipedia. https://en.wikipedia.org/wiki/Machine_vision
- “Digital image processing,” Wikipedia. https://en.wikipedia.org/wiki/Digital_image_processing
- “Training Convolutional Neural Networks: What Is Machine Learning” Analog Devices.